

PROJECT TITLE

CashSplash: Saving Companion Modernizing Budget Payment
(Budget tracker)

SEMESTER VI

IN

INFORMATION TECHNOLOGY

BY

Licia Almeida

Janaki Bal

Shravani Jadhav

Priyadarshini Sandiliyan

UNDER THE GUIDANCE OF

Ms. Stella J.

(Assistant Professor, Department of Information Technology)



INFORMATION TECHNOLOGY DEPARTMENT

XAVIER INSTITUTE OF ENGINEERING

UNIVERSITY OF MUMBAI

2024-2025



XAVIER INSTITUTE OF ENGINEERING

Department of Information Technology

(NBA Accredited)

(Approved by AICTE, Accredited with "A+" by NAAC, Affiliated to Mumbai University,

Recognized by DTE, Govt. of Maharashtra)

Mahim Causeway, Mahim, Mumbai - 400 016 • Phone : +91 7249371505 / +91 9920407274

Email ID : hod-it@xavier.ac.in • Website : www.xavier.ac.in

CERTIFICATE

This to certify that

LICIA ALMEIDA	(202203005)
SHRAVANI JADHAV	(202203028)
JANAKI BAL	(202203007)
PRIYADARSHINI SANDILYAN	(202203004)

Have satisfactorily carried out the MAD AND PWA MINI-PROJECT work titled "CashSplash" in partial fulfillment of the degree of Bachelor of Engineering as laid down by the University of Mumbai during the academic year 2024-2025.

Internal Examiner/Guide

External Examiner

Date:

Place: MAHIM, MUMBAI

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' Ideas or words have been included, I have adequately cited and referenced the original sources.

I also declare that I have adhered to all the principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission.

I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which thus have not been properly cited or from whom proper permission have not been taken when needed.

LICIA ALMEIDA (202203005) -----

JANAKI BAL (202203007) -----

SHRAVANI JADHAV (202203028) -----

PRIYADARSHINI SANDILYAN (202203004) -----

Date:

TABLE OF CONTENTS

SR.NO	TOPIC	PAGE NO.
A.	LIST OF FIGURES	i
B.	LIST OF TABLES	ii
C.	ABSTRACT	iii
D.	ACKNOWLEDGEMENT	iv
E.	LAB OUTCOMES	v
1.	INTRODUCTION TO MOBILE APPLICATION DEVELOPMENT	1
2.	INTRODUCTION TO THE SELECTED MINI-PROJECT TOPIC (LO3, LO6) 2.1 PROBLEM DEFINITION 2.2 AIMS AND OBJECTIVES 2.3 SCOPE OF THE PROJECT	2
3.	SYSTEM DESCRIPTION (LO3, LO6) 3.1 SYSTEM DESIGN 3.2 REQUIREMENT OF HARDWARE & SOFTWARE 3.3 IMPLEMENTATION METHODOLOGY 3.4 CODE 3.5 FINAL PROTOTYPE 3.6 CONCLUSION & FUTURE SCOPE OF THE PROJECT 3.7 CONSTRAINTS FOR REAL TIME DEPLOYMENT	3-53 3-8 8 8-9 9-49 50-52 53 53
4.	REFERENCES	54

LIST OF FIGURES

SR NO.	FIGURE CAPTION	PAGE NO.
1	Flow Chart	7
2	Block Diagram	8
3	Expense List	50
4	Home Screen	50
5	Profile Edit page	51
6	Login Page	51
7	Expense Entry Page	52
8	Exceeding Budget Notification	52

SR NO.	TABLE TITLE	PAGE NO.
1	Hardware, Software and Cloud Platforms Used	8

The digital transformation of personal finance management is significantly reshaping how individuals monitor expenses, create budgets, and pursue their financial goals. “CashSplash – Savings Companion” is an innovative, mobile-based personal finance application designed to provide users with greater control over both their daily expenditures and long-term money management strategies. Featuring real-time expense tracking, customizable budget planning tools, and intuitive savings goal visualization, CashSplash empowers users to make smarter and more informed financial decisions. Built using Firebase for enhanced backend scalability, real-time data syncing, and offline functionality, the app ensures a seamless experience even in low-connectivity scenarios. With a clean, user-friendly interface, secure authentication, detailed transaction logging, and insightful expense breakdowns, CashSplash modernizes traditional budgeting methods. The application also encourages responsible spending behavior and promotes the development of healthy financial habits by leveraging the power of modern technology.

We would like to thank Fr. Dr. John Rose S.J (Director of XIE) for providing us with such an environment so as to achieve goals of our project and supporting us constantly.

We express our sincere gratitude to our Honorable Principal Dr. Y.D.Venkatesh for encouragement and facilities provided to us.

We would like to place on record our deep sense of gratitude to Dr. Jaychand Upadhyay, Head of Dept Of Information Technology, Xavier Institute of Engineering, Mahim, Mumbai, for her generous guidance help and useful suggestions.

With deep sense of gratitude we acknowledge the guidance of our project guide **Ms. Stella J.** The time-to-time assistance and encouragement by her has played an important role in the development of our project.

We would also like to thank our entire Information Technology staff who have willingly cooperated with us in resolving our queries and providing us all the required facilities on time.

LICIA ALMEIDA

JANAKI BAL

SHRAVANI JADHAV

PRIYADARSHINI SANDILYAN



XAVIER INSTITUTE OF ENGINEERING

Department of Information Technology

(NBA Accredited)

(Approved by AICTE, Accredited with "A+" by NAAC, Affiliated to Mumbai University,
Recognized by DTE, Govt. of Maharashtra)

Mahim Causeway, Mahim, Mumbai - 400 016 • Phone : +91 7249371505 / +91 9920407274

Email ID : hod-it@xavier.ac.in • Website : www.xavier.ac.in

TE (IT) SEM-VI (AY: 2024-25)

Subject: MAD & PWA Lab (Lab code:ITL604)

Course Code	Course Name	Teaching Scheme (Contact Hours)		Credits Assigned		
		Theory	Practical	Theory	Practical	Total
ITL604	MAD & PWA Lab	--	02	--	01	01

Course Code	Course Name	Examination Scheme							
		Theory					Term Work	Pract / Oral	Total
		Internal Assessment			End Sem Exam	Exam Duration (in Hrs)			
		Test1	Test 2	Avg					
ITL604	MAD & PWA Lab	--	--	--	--	- -	25	25	50

Lab Objectives:

1. To learn To learn the basics of Flutter framework.
2. Develop the App UI by incorporating widgets, layouts, gestures and animation.
3. Create a production ready Flutter App by including files and Firebase back end service.
4. Learn the Essential technologies, and Concepts of PWA's to get started as quickly and efficiently as possible.
5. Develop responsive web applications by combining AJAX development techniques with the jQuery JavaScript library.
6. Understand how service workers operate and also learn to Test and Deploy PWA.

Lab Outcomes:

LO	LO Statement	PO Mapped
1	Design and Develop cross platform mobile application development using Flutter framework	PO1, PO3, PO5, PO6, PO8, PO9, PO10, PO12
2	Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation	PO1, PO3, PO5, PO6, PO8, PO9, PO10, PO12
3	Analyze and Build production ready Flutter App by incorporating back end services and deploying on Android / iOSBuild	PO1, PO3, PO5, PO6, PO8, PO9, PO10, PO12
4	Analyze various PWA frameworks and their requirements	PO1, PO3, PO5, PO6, PO8, PO9, PO10, PO12
5	Design and Develop a responsive User Interface by applying PWA Design techniques	PO1, PO3, PO5, PO6, PO8, PO9, PO10, PO12
6	Develop and Analyze PWA Features and deploy it over app hosting solutions	PO1, PO3, PO5, PO6, PO8, PO9, PO10, PO12



XAVIER INSTITUTE OF ENGINEERING

Department of Information Technology

(NBA Accredited)

(Approved by AICTE, Accredited with "A+" by NAAC, Affiliated to Mumbai University,
Recognized by DTE, Govt. of Maharashtra)

Mahim Causeway, Mahim, Mumbai - 400 016 • Phone : +91 7249371505 / +91 9920407274

Email ID : hod-it@xavier.ac.in • Website : www.xavier.ac.in

COURSE OUTCOMES

CO No.	COURSE OUTCOME
ITL 604.1	Design and Develop cross platform mobile application development using Flutter framework
ITL 604.2	Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
ITL 604.3	Analyze and Create production ready Flutter App by incorporating back end services and deploying on Android / iOSBuild
ITL 604.4	Analyze various PWA frameworks and their requirements
ITL 604.5	Design and Develop a responsive User Interface by applying PWA Design techniques
ITL 604.6	Develop and Analyze PWA Features and deploy it over app hosting solutions

CO-PO-PSO MAPPING

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
LO1	3	1			3	3		3	3	2		2	1	1
LO2	3	1	2		3	3		3	3	2		2	3	3
LO3	3	1	2		3	3		3	3	2		2	3	3
LO4	3	1			3	3		3	3	2		2	1	1
LO5	3	1	2		3	3		3	3	2		2	3	3
LO6	3	1	2		3	3		3	3	2		2	3	3



XAVIER INSTITUTE OF ENGINEERING

Department of Information Technology

(NBA Accredited)

(Approved by AICTE, Accredited with "A+" by NAAC, Affiliated to Mumbai University,
Recognized by DTE, Govt. of Maharashtra)

Mahim Causeway, Mahim, Mumbai - 400 016 • Phone : +91 7249371505 / +91 9920407274

Email ID : hod-it@xavier.ac.in • Website : www.xavier.ac.in

Class/ Sem/ A.Y: TE IT/ VI/ 2024-2025

Course Name: MAD & PWA Lab [ITL604]

Group No/ Roll No:

Mini Project

- LO1: Design and Develop** cross platform mobile application development using Flutter framework
LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
LO3: Analyze and Build production ready Flutter App by incorporating back end services and deploying on Android / iOSBuild
LO4: Analyze various PWA frameworks and their requirements
LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
LO6: Develop and Analyze PWA Features and deploy it over app hosting solutions

Rubrics for Mini Project Work

Roll No.	Name of the Student	Problem Statement (5)	Creativity & Quality of Work done (04)	Punctuality & lab ethics (02)	Performance/ Presentation (04)	Total (15)

Prof. Stella J
Subject Incharge

1. Introduction to Mobile Application Development (MAD)

Mobile Application Development (MAD) is the process of designing, building, and deploying software applications that run on mobile devices such as smartphones and tablets. With the increasing penetration of mobile technology in everyday life, MAD has become a crucial area in modern software engineering. It involves selecting suitable platforms (Android, iOS), development tools, and frameworks to create responsive, interactive, and user-friendly applications. MAD also emphasizes performance, scalability, and seamless user experience. Key components include user interface design, backend integration, database management, and device hardware utilization. This field continues to evolve with the rise of cross-platform tools and progressive web apps (PWAs).

2. Introduction to Selected Mini-Project Topic

2.1 Problem Definition

In today's fast-paced digital era, managing personal finances efficiently has become a growing challenge. Many individuals still rely on manual methods such as notebooks or basic spreadsheets to track expenses and savings, which are error-prone and lack real-time accessibility. The absence of automated tracking, categorized budgeting, and visualization tools often leads to overspending and poor savings discipline. Existing financial applications may not offer personalized features, offline access, or intuitive design for diverse users. Thus, there is a need for a modern, accessible, and intelligent mobile solution that empowers users to monitor, plan, and control their finances effectively.

2.2 Aims and Objectives

- To design and develop a mobile application named **CashSplash – Savings Companion** for personal finance management.
- To enable users to track daily expenses, plan monthly budgets, and set long-term savings goals.
- To implement features such as secure user login, receipt viewing, and expense breakdown visualization.
- To integrate Firebase for real-time backend support and provide offline functionality for uninterrupted usage.
- To deliver a simple and user-friendly interface that supports financial literacy and planning.

2.3 Scope of the Project

The scope of this project includes the development of an Android-based mobile application focused on personal budgeting and expense management. The application will cover features such as user authentication, transaction entry, budget monitoring, savings goal tracking, and expense categorization. It will support both online and offline modes and use Firebase as the backend database for scalability. The scope also extends to user interface design, functionality testing, and consideration of future enhancements like AI-based financial advice and shared budgeting. This project does not include financial transactions or integration with external banking APIs in its current scope.

3. System Description

3.1 Design:

Flowchart Explanation

The flowchart consists of six primary stages in the app's navigation:

1. Login Screen

- The entry point where existing users log in using Firebase Authentication.
- Navigation proceeds to Home or Create User.

2. Create User

- First-time users to register by entering basic details.
- Once done, it redirects to Home.

3. Home Dashboard

- Central hub displaying current budget status, summary of expenses, and navigation options to other modules.

4. Track Expenses

- Users can input daily expenses with amount, category.
- Data is stored in Firebase Realtime Database.

5. Edit Profile

- Allows updating user information and resetting credentials if needed.

6. View Budget

- Visual graphs and budget targets displayed using data fetched from Firebase.
- Encourages savings by showing how much money is left and notifies on crossing a Threshold.

GUI Screenshots:



[Home](#)
[About Us](#)
[Service](#)
[Contact](#)

LOGIN

- First Screen is Login screen
- User enters ID and Password
- He can also navigate to create a new user

[Home](#)
[About Us](#)
[Service](#)
[Contact](#)

CREATE USER

- User can give credentials such as user name, email ID and create a password
- It checks for uniqueness of the email ID from the database and creates user accounts

[Home](#)
[About Us](#)
[Service](#)
[Contact](#)

Edit PROFILE

- Can Contain Personal Information
- It also give us the option to modify Monthly Budget Goal

← → ↺ localhost:58120

⚙️ ☆ 👤 ⋮

←

Profile

DEMO



Personal Information

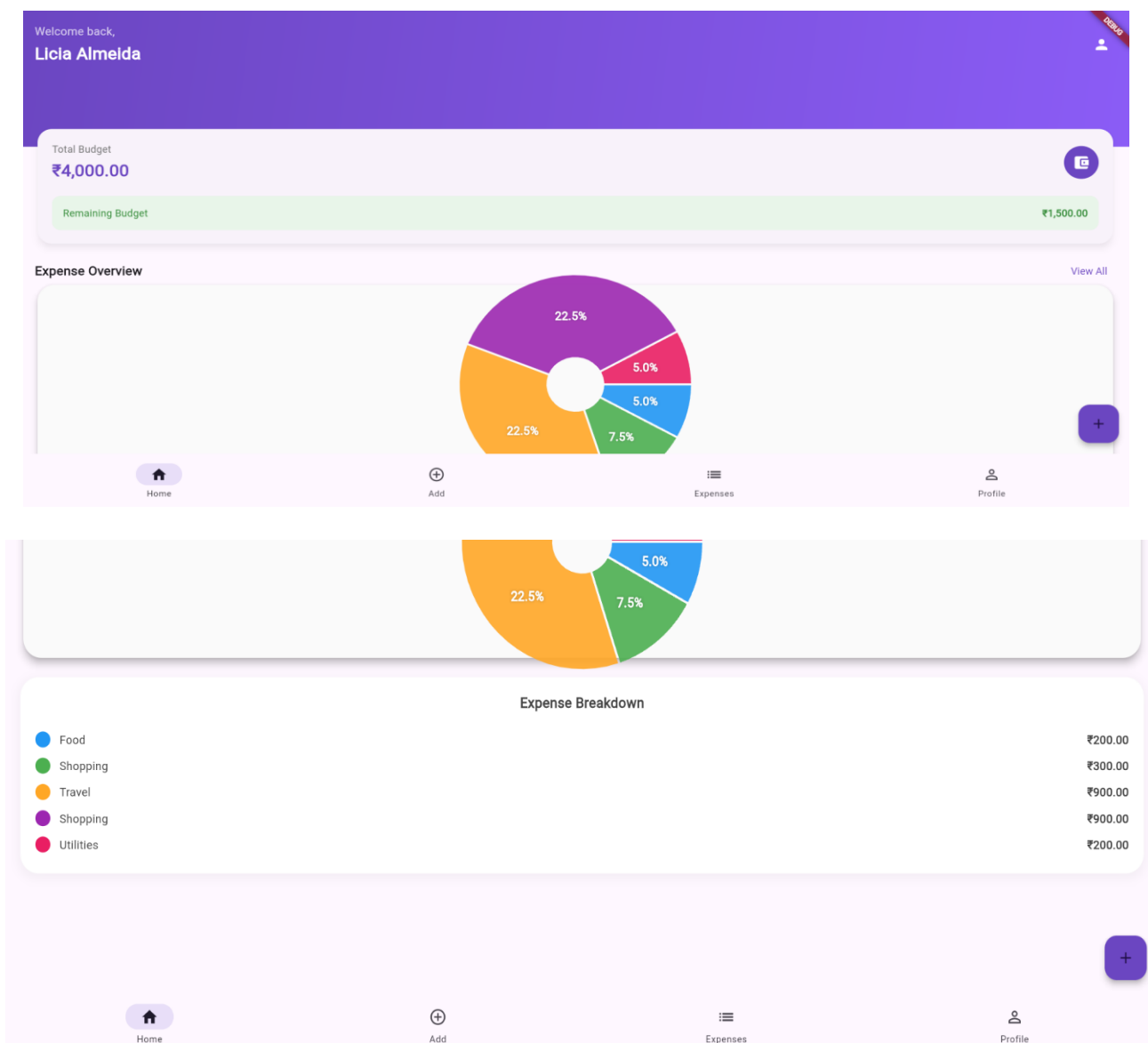
Name

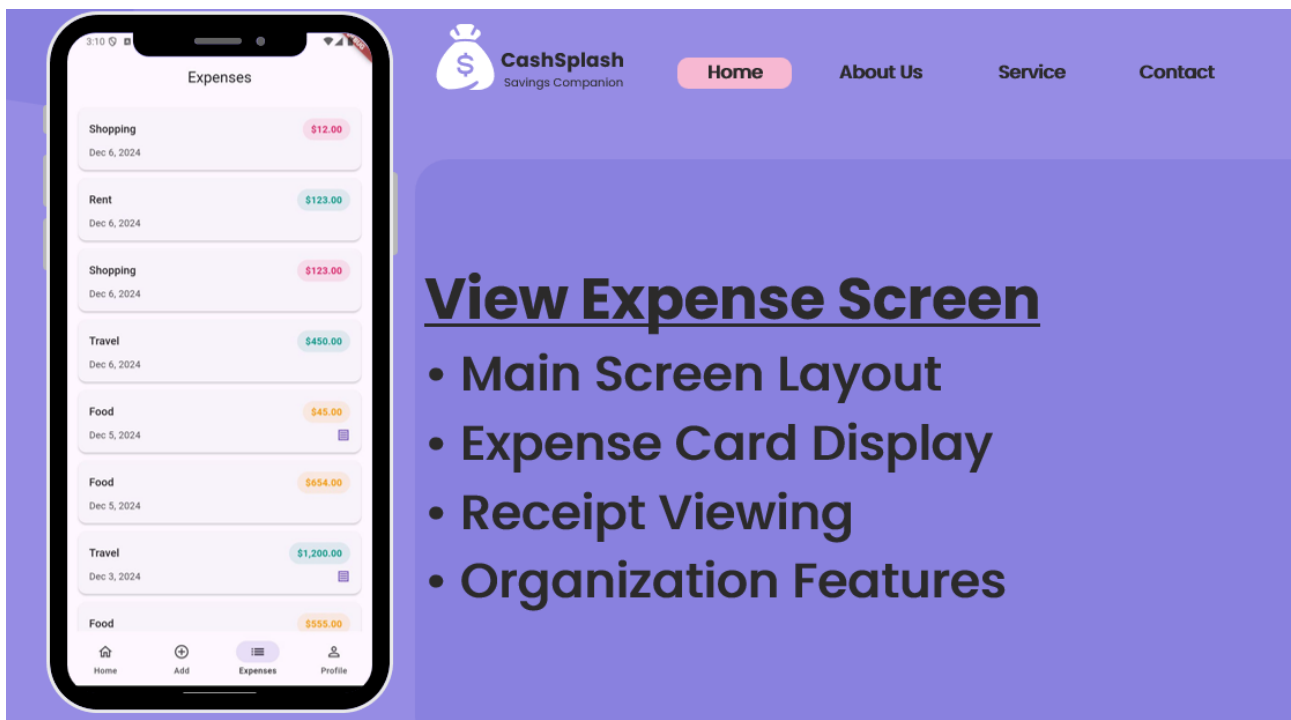
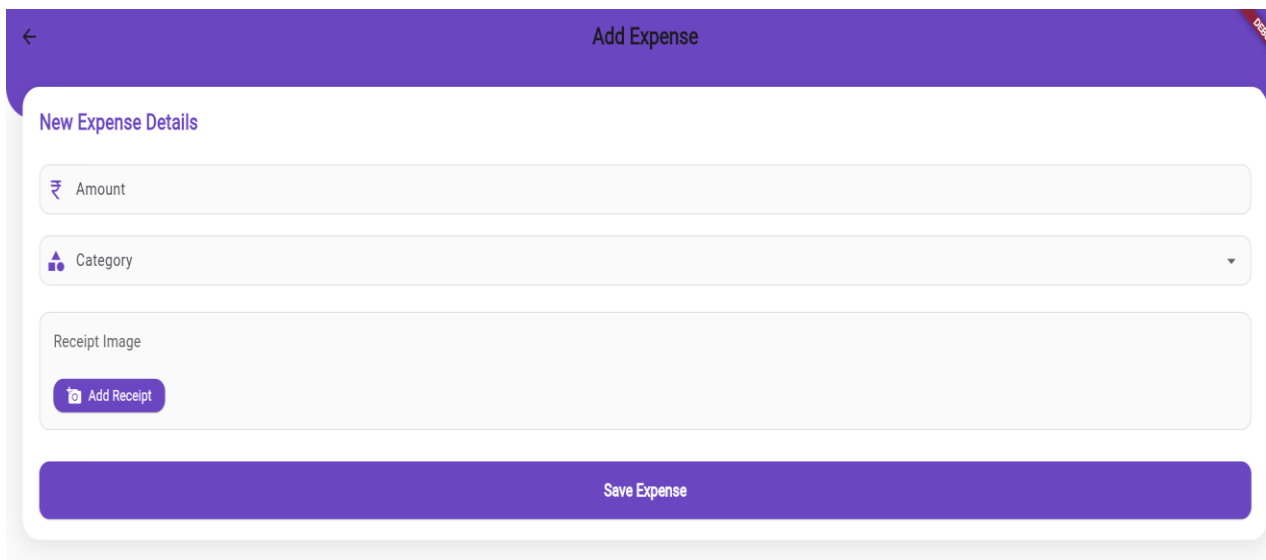
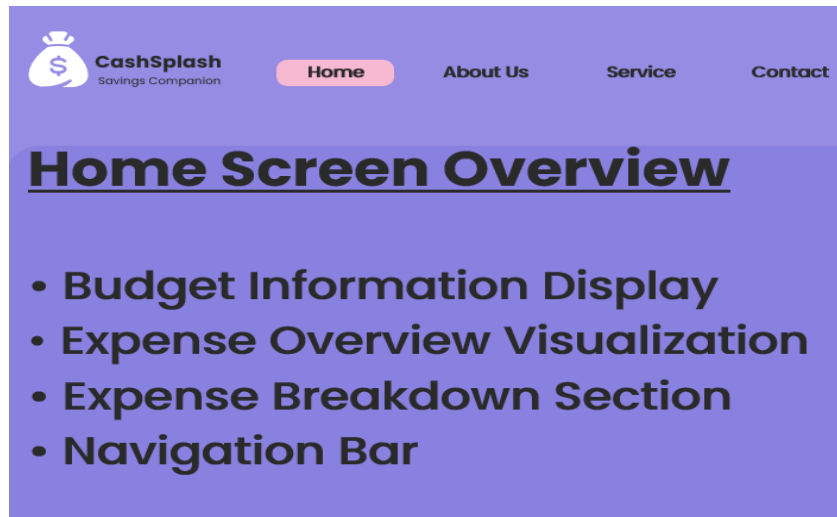
Janaki Bal

Monthly Budget Goal

₹ 2000

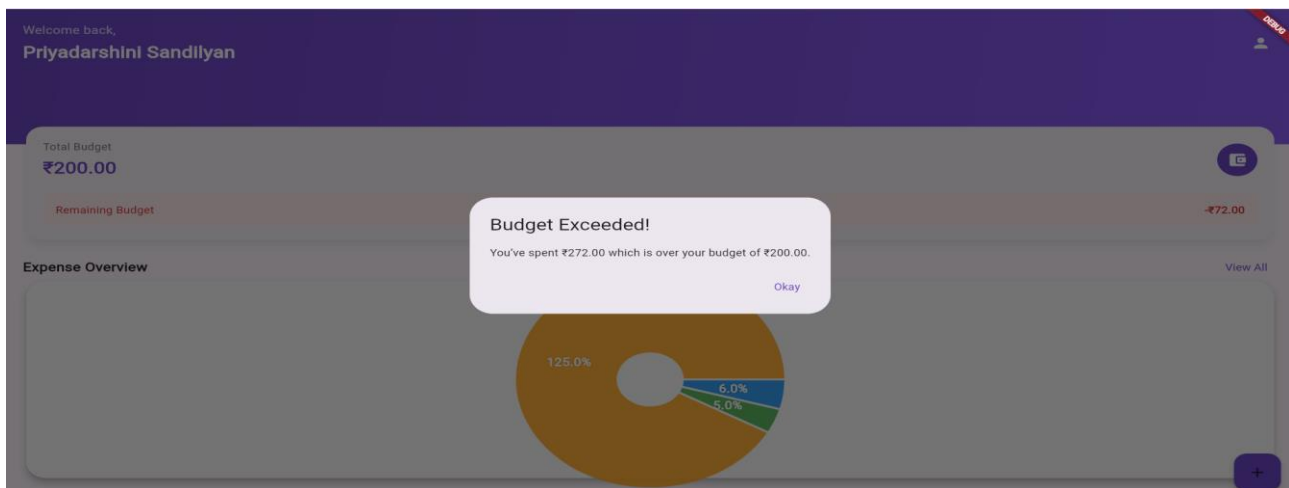
Save Changes



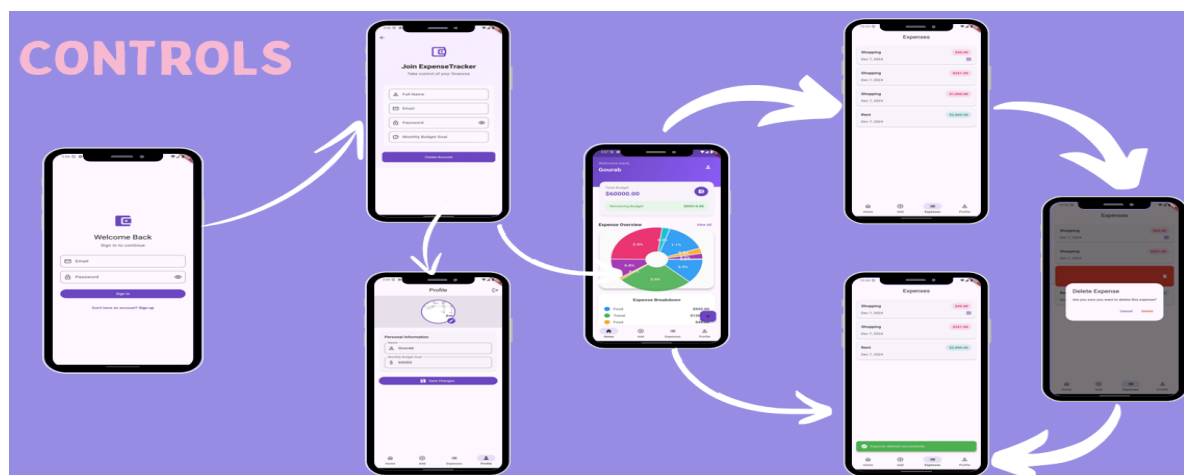


Expenses	
Utilities Apr 16, 2025	₹200.00
Shopping Apr 16, 2025	₹900.00
Travel Apr 16, 2025	₹900.00
Shopping Apr 16, 2025	₹300.00
Food Apr 16, 2025	₹200.00

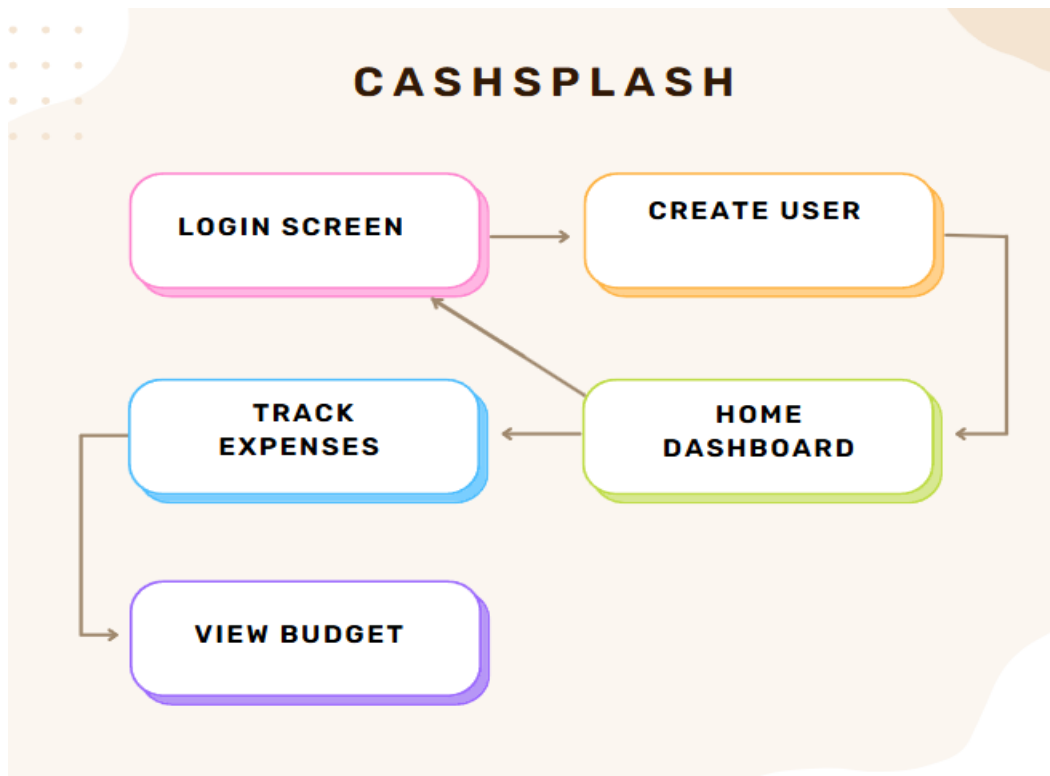
Notification For Exceeding Budget:



2) Flow chart: (Fig 1)



3) Block Diagram:(Fig 2)



The design of **CashSplash – Savings Companion** is structured around smooth user navigation and modular functionality to ensure seamless financial tracking. The application starts with user authentication and flows through various essential modules like expense tracking, savings goals, and budget overview.

3.2 Hardware, Software and Cloud Platforms Used (Table 1)

Component	Specification
Device	Android smartphone
OS	Android 10+
IDE	Android Studio
Language	Dart (using Flutter framework)
Database	Firebase Realtime Database
Authentication	Firebase Auth
Design Tools	Dart Widgets (Flutter UI Components)

3.3 Implementation Methodology

To solve the problem of manual expense tracking and budgeting, we developed a mobile application using Flutter and Firebase. The implementation was done in the following steps:

1. **Firestore Setup:**

A Firestore project was created and integrated with the Flutter app to provide backend support for user authentication and real-time database operations.

2. **User Authentication:**

Login and Sign-Up screens were implemented using Firestore Authentication, allowing users to securely register and access their accounts.

3. **User Interface Design:**

All screens, such as Home, Add Expense, View Expenses, and Profile Settings, were designed using Flutter widgets. This ensured a smooth, responsive, and platform-independent UI.

4. **Expense Management:**

Users can log their daily expenses, which are stored in Firestore Realtime Database under their unique user ID. The app fetches and displays this data in real time.

5. **Budget Goal & Visualization:**

Users can set a monthly budget goal. The app calculates the remaining budget and visualizes spending using pie charts and percentage indicators for better clarity.

6. **Receipt Upload & Viewing:**

Users can optionally upload receipts for expenses as base64-encoded images, which can be viewed later directly from the app.

7. **Navigation & User Flow:**

Easy navigation between screens was implemented using Flutter's Navigator and page transitions. This ensures users can access key features quickly.

8. **Notification System:**

Budget notifications were added using `flutter_local_notifications` to alert users when they approach their spending limits, promoting better financial awareness.

3.4 Code

main.dart:

```
import
'package:firebase_auth/firebase_auth.dart';
import
'package:firebase_core/firebase_core.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import
'package:project/profile_settings_screen.dart';
import
'package:project/view_expenses_screen.dart';
import 'add_expense_screen.dart';
import 'home_screen.dart';
import 'login_screen.dart';

import 'firebase_options.dart';

class SplashScreen extends StatefulWidget {
  @override
  _SplashScreenState createState() =>
    _SplashScreenState();
```

```

}

class _SplashScreenState extends
State<SplashScreen> with
SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<double> _scaleAnimation;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      duration: const Duration(seconds: 2),
      vsync: this,
    )..repeat(reverse: true);

    _scaleAnimation = Tween<double>(begin:
1.0, end: 1.1).animate(
      CurvedAnimation(parent: _controller,
curve: Curves.easeInOut),
    );

    _initializeApp();
  }

  Future<void> _initializeApp() async {

    await Future.delayed(Duration(seconds:
3));
    if (mounted) {
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder: (context) =>
AuthenticationWrapper()),
      );
    }
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Color(0xFF6B46C1),
      body: Center(

```

```

child: Column(
  mainAxisAlignment:
MainAxisAlignment.center,
  children: [
    ScaleTransition(
      scale: _scaleAnimation,
      child: Image.asset(
        'assets/Picture1.png',
        width: 120,
        height: 120,
      ),
    ),
    SizedBox(height: 24),
    Text(
      'CashSplash',
      style: TextStyle(
        fontSize: 32,
        fontWeight: FontWeight.bold,
        color: Color(0xFFA78BFA),
      ),
    ),
    Text(
      'Savings Companion',
      style: TextStyle(
        fontSize: 18,
        color:
Color(0xFFA78BFA).withOpacity(0.8),
      ),
    ),
  ],
),
);
}

void main() async{
  await Firebase.initializeApp(
    options:
DefaultFirebaseOptions.currentPlatform,
  );
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'CashSplash',

```

```

theme: ThemeData(
  useMaterial3: true,
  colorScheme: ColorScheme.fromSeed(
    seedColor: Color(0xFF6B46C1),
    primary: Color(0xFF6B46C1),
    secondary: Color(0xFFFFE4E1),
  ),
  inputDecorationTheme:
InputDecorationTheme(
  border: OutlineInputBorder(
    borderRadius:
BorderRadius.circular(12),
  ),
),
),
home: SplashScreen(),
);
}
}

```

```

class AuthenticationWrapper extends
StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return StreamBuilder<User?>(
      stream:
FirebaseAuth.instance.authStateChanges(),
      builder: (context, snapshot) {
        if (snapshot.connectionState ==
ConnectionState.active) {
          User? user = snapshot.data;
          if (user == null) return LoginScreen();
          return MainAppScreen();
        }
        return Scaffold(
          body: Center(child:
CircularProgressIndicator()),
        );
      },
    );
  }
}

```

```

class MainAppScreen extends StatefulWidget

```

```

{
  @override
  _MainAppScreenState createState() =>
_MainAppScreenState();
}

```

```

class _MainAppScreenState extends
State<MainAppScreen> {
  int _selectedIndex = 0;
  final List<Widget> _pages = [
    HomeScreen(),
    AddExpenseScreen(),
    ViewExpensesScreen(),
    ProfileSettingsScreen(),
  ];

```

```

void _onItemTapped(int index) {
  setState(() => _selectedIndex = index);
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: IndexedStack(
      index: _selectedIndex,
      children: _pages,
    ),
    bottomNavigationBar: NavigationBar(
      selectedIndex: _selectedIndex,
      onDestinationSelected: _onItemTapped,
      backgroundColor:
Theme.of(context).colorScheme.surface,
      destinations: const [
        NavigationDestination(
          icon: Icon(Icons.home_outlined),
          selectedIcon: Icon(Icons.home),
          label: 'Home',
        ),
        NavigationDestination(
          icon: Icon(Icons.add_circle_outline),
          selectedIcon: Icon(Icons.add_circle),
          label: 'Add',
        ),
        NavigationDestination(
          icon: Icon(Icons.list_outlined),
          selectedIcon: Icon(Icons.list),
          label: 'Expenses',
        ),
      ],
    ),
  );
}

```

```

),
  NavigationDestination(
    icon: Icon(Icons.person_outline),
    selectedIcon: Icon(Icons.person),
    label: 'Profile',
  ),
],
),
);
}
}

```

add_expense_screen.dart

```

import 'package:flutter/material.dart';
import
'package:firebase_database/firebase_database.
dart';
import
'package:firebase_auth/firebase_auth.dart';
import
'package:image_picker/image_picker.dart';
import 'dart:io';
import 'dart:convert'; // For base64 encoding
import 'home_screen.dart';

```

```

class AddExpenseScreen extends
StatefulWidget {
  @override
  _AddExpenseScreenState createState() =>
  _AddExpenseScreenState();
}

```

```

class _AddExpenseScreenState extends
State<AddExpenseScreen> {
  final _formKey = GlobalKey<FormState>();
  TextEditingController _amountController =
TextEditingController();
  String? _selectedCategory;
  final categories = ['Food', 'Travel', 'Rent',
'Shopping', 'Utilities', 'Other'];
  File? _receiptImage;
  final picker = ImagePicker();

```

```

// Method to convert image to base64 string

```

```

Future<String> _imageToBase64(File
imageFile) async {
  List<int> imageBytes = await
imageFile.readAsBytes();
  return base64Encode(imageBytes);
}

```

```

@override
void dispose() {
  _amountController.dispose();
  super.dispose();
}

```

```

Future<void> _pickImage() async {
  final pickedFile = await
picker.pickImage(source:
ImageSource.gallery);
  if (pickedFile != null) {
    setState() {
      _receiptImage = File(pickedFile.path);
    });
  }
}

```

```

void _clearForm() {
  _amountController.clear();
  setState() {
    _selectedCategory = null;
    _receiptImage = null;
  });
}

```

```

FocusScope.of(context).requestFocus(FocusN
ode());
}

```

```

Future<bool> _onWillPop() async {
  if (_amountController.text.isNotEmpty ||
    _selectedCategory != null ||
    _receiptImage != null) {
    return await showDialog(
      context: context,
      builder: (context) => AlertDialog(
        title: Text('Discard Changes?'),
        content: Text('You have unsaved
changes. Are you sure you want to leave?'),
        actions: [
          TextButton(
            onPressed: () =>

```

```

Navigator.of(context).pop(false),

child: Text('Stay'),
),
TextButton(
  onPressed: () =>
Navigator.of(context).pop(true),
  child: Text('Leave'),
  style:
TextButton.styleFrom(foregroundColor:
Colors.red),
),
),
) ??
false;
}
return true;
}

Future<void> _saveExpense() async {

  if (!_formKey.currentState!.validate())
return;

  try {
    // Convert receipt image to base64 if
exists
    String? base64Image;
    if (_receiptImage != null) {
      base64Image = await
_imageToBase64(_receiptImage!);
    }

    // Ensure the user is logged in
    final user =
FirebaseAuth.instance.currentUser;
    if (user == null) {
      throw Exception("User not logged in");
    }

    // Build expense data without null fields
    final Map<String, dynamic> expenseData
= {
      'amount': _amountController.text,
      'category': _selectedCategory,
      'date':
DateTime.now().toIso8601String(),

```

```

    };
    if (base64Image != null &&
base64Image.isNotEmpty) {

      expenseData['receiptImage'] =
base64Image;
    }

    // Create the DatabaseReference using the
valid user.uid
    DatabaseReference dbRef =
FirebaseDatabase.instance.ref("expenses/${us
er.uid}");
    await dbRef.push().set(expenseData);

    if (!context.mounted) return;

ScaffoldMessenger.of(context).showSnackBa
r(
  SnackBar(
    content: Row(
      children: [
        Icon(Icons.check_circle, color:
Colors.white),
        SizedBox(width: 8),
        Text('Expense added successfully'),
      ],
    ),
    backgroundColor: Colors.green,
    behavior: SnackBarBehavior.floating,
    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
    duration: Duration(seconds: 2),
    action: SnackBarAction(
      label: 'Add Another',
      textColor: Colors.white,
      onPressed: () {
        _clearForm();
      },
    ),
  ),
);

_clearForm();

} catch (error) {
  if (!context.mounted) return;

```

```
ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
```

```
    content: Row(
      children: [
        Icon(Icons.error_outline, color:
Colors.white),
        SizedBox(width: 8),
        Expanded(child: Text('Failed to save
expense: $error')),
      ],
    ),
    backgroundColor: Colors.red,
    behavior: SnackBarBehavior.floating,
    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
  ),
);
}
}
```

```
@override
Widget build(BuildContext context) {
  return WillPopScope(
    onWillPop: _onWillPop,
    child: Scaffold(
      backgroundColor: Colors.grey[50],
      appBar: AppBar(
        elevation: 0,
        backgroundColor:
Color(0xFF6B46C1),
        title: Text('Add Expense', style:
TextStyle(fontWeight: FontWeight.w600)),
        centerTitle: true,
        leading: IconButton(
          icon: Icon(Icons.arrow_back),
          onPressed: () async {
            if (await _onWillPop()) {
              if (context.mounted) {
                Navigator.of(context).pop();
              }
            }
          },
        ),
      ),
      body: SingleChildScrollView(
        child: Column(
```

```
        children: [
          Container(
            decoration: BoxDecoration(
              color: Color(0xFF6B46C1),
              borderRadius: BorderRadius.only(
```

```
              bottomLeft: Radius.circular(30),
              bottomRight:
Radius.circular(30),
            ),
            height: 50,
          ),
          Transform.translate(
            offset: Offset(0, -30),
            child: Container(
              margin:
EdgeInsets.symmetric(horizontal: 20),
              decoration: BoxDecoration(
                color: Colors.white,
                borderRadius:
BorderRadius.circular(20),
                boxShadow: [
                  BoxShadow(
                    color:
Colors.black.withOpacity(0.1),
                    blurRadius: 20,
                    offset: Offset(0, 5),
                  ),
                ],
              ),
              child: Padding(
                padding: EdgeInsets.all(20),
                child: Form(
                  key: _formKey,
                  child: Column(
                    crossAxisAlignment:
CrossAxisAlignment.stretch,
                    children: [
                      Text(
                        'New Expense Details',
                        style: TextStyle(
                          fontSize: 20,
                          fontWeight:
FontWeight.bold,
                          color:
Color(0xFF6B46C1),
                        ),
                      ),
                      SizedBox(height: 25),
```

```

        TextFormField(
          controller:
            _amountController,
          decoration:
            InputDecoration(
              labelText: 'Amount',
              prefixIcon:

Icon(Icons.currency_rupee, color:
Color(0xFF6B46C1)),
              border:
            OutlineInputBorder(
              borderRadius:
                BorderRadius.circular(12),
              borderSide:
                BorderSide(color: Colors.grey.shade300),
              enabledBorder:
            OutlineInputBorder(
              borderRadius:
                BorderRadius.circular(12),
              borderSide:
                BorderSide(color: Colors.grey.shade300),
              focusedBorder:
            OutlineInputBorder(
              borderRadius:
                BorderRadius.circular(12),
              borderSide:
                BorderSide(color: Color(0xFF6B46C1)),
              filled: true,
              fillColor: Colors.grey[50],
            ),
              keyboardType:
                TextInputType.numberWithOptions(decimal:
true),
              validator: (value) {
                if (value == null ||
value.isEmpty) return 'Please enter amount';
                if (double.tryParse(value)
== null) return 'Please enter a valid number';
                return null;
              },
            ),
            SizedBox(height: 20),

DropdownButtonFormField<String>(
          value: _selectedCategory,
          decoration:
            InputDecoration(

```

```

              labelText: 'Category',
              prefixIcon:
            Icon(Icons.category, color:
Color(0xFF6B46C1)),
              border:
            OutlineInputBorder(
              borderRadius:

                BorderRadius.circular(12),
              borderSide:
                BorderSide(color: Colors.grey.shade300),
              enabledBorder:
            OutlineInputBorder(
              borderRadius:
                BorderRadius.circular(12),
              borderSide:
                BorderSide(color: Colors.grey.shade300),
              focusedBorder:
            OutlineInputBorder(
              borderRadius:
                BorderRadius.circular(12),
              borderSide:
                BorderSide(color: Color(0xFF6B46C1)),
              filled: true,
              fillColor: Colors.grey[50],
            ),
              items:
                categories.map((category) {
                  return
                    DropdownMenuItem(
                      value: category,
                      child: Text(category),
                    );
                }).toList(),
              onChanged: (value) =>
                setState(() => _selectedCategory = value),
              validator: (value) => value
                == null ? 'Please select a category' : null,
            ),
            SizedBox(height: 25),
            Container(
              decoration: BoxDecoration(
                color: Colors.grey[50],
                borderRadius:
                  BorderRadius.circular(12),
                border: Border.all(color:
Colors.grey.shade300),

```



```

    ],
  ),
),
);
}
}

```

```

    _scaleAnimation = Tween<double>(begin:
1.0, end: 1.2).animate(
    CurvedAnimation(parent: _controller,
curve: Curves.easeInOut),
    );

_initializeApp();
}

```

```

Future<void> _initializeApp() async {
  if (kIsWeb) {

```

animated_splash_screen.dart

```

import
'package:firebase_auth/firebase_auth.dart';
import
'package:firebase_core/firebase_core.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import
'package:project/profile_settings_screen.dart';
import
'package:project/view_expenses_screen.dart';
import 'add_expense_screen.dart';
import 'home_screen.dart';
import 'login_screen.dart';

```

```

class SplashScreen extends StatefulWidget {
  @override
  _SplashScreenState createState() =>
_SplashScreenState();
}

```

```

class _SplashScreenState extends
State<SplashScreen> with
SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<double> _scaleAnimation;

```

```

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      duration: const Duration(seconds: 2),
      vsync: this,
    )..repeat(reverse: true);

```

```

await Firebase.initializeApp(
  options: const FirebaseOptions(
    apiKey:
"AIzaSyD2ZDdfmv0FtJ9134LDRPFf4ezV12
pMQGg",
    authDomain: "cashsplash-
774ca.firebaseio.com",
    projectId: "cashsplash-774ca",
    storageBucket: "cashsplash-
774ca.firebaseioapp",
    messagingSenderId: "1079637643425",
    appId:
"1:1079637643425:web:bdeaeaa5ac37ba9da3
20e0",
    measurementId: "G-8CYY8EZ7FV",
  ),
);
} else {
  await Firebase.initializeApp();
}

```

```

await Future.delayed(Duration(seconds:
3));
if (mounted) {
  Navigator.pushReplacement(
    context,
    MaterialPageRoute(builder: (context) =>
AuthenticationWrapper()),
  );
}
}

```

```

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();

```

```

}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Color(0xFFFFFE4E1),
    body: Center(
      child: ScaleTransition(
        scale: _scaleAnimation,
        child: Image.asset(
          'assets/app_icon.png',
          width: 150,
          height: 150,
        ),
      ),
    ),
  );
}

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'CashSplash',
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.blue,
          secondary: Colors.green,
        ),
        inputDecorationTheme:
InputDecorationTheme(
          border: OutlineInputBorder(
            borderRadius:
BorderRadius.circular(12),
          ),
        ),
        home: SplashScreen(),
      );
    }
  }
}

```

```

class AuthenticationWrapper extends
StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return StreamBuilder<User?>(
      stream:
FirebaseAuth.instance.authStateChanges(),
      builder: (context, snapshot) {
        if (snapshot.connectionState ==
ConnectionState.active) {
          User? user = snapshot.data;
          if (user == null) return LoginScreen();
          return MainAppScreen();
        }
        return Scaffold(
          body: Center(child:

```

```

CircularProgressIndicator()),
        );
      },
    );
  }
}

```

```

class MainAppScreen extends StatefulWidget
{
  @override
  _MainAppScreenState createState() =>
_MainAppScreenState();
}

```

```

class _MainAppScreenState extends
State<MainAppScreen> {
  int _selectedIndex = 0;
  final List<Widget> _pages = [
    HomeScreen(),
    AddExpenseScreen(),
    ViewExpensesScreen(),
    ProfileSettingsScreen(),
  ];

  void _onItemTapped(int index) {
    setState(() => _selectedIndex = index);
  }
}

```

```

@override
Widget build(BuildContext context) {

```

```

return Scaffold(
  body: IndexedStack(
    index: _selectedIndex,
    children: _pages,
  ),
  bottomNavigationBar: NavigationBar(
    selectedIndex: _selectedIndex,
    onDestinationSelected: _onItemTapped,
    destinations: const [
      NavigationDestination(
        icon: Icon(Icons.home_outlined),
        selectedIcon: Icon(Icons.home),
        label: 'Home',
      ),
      NavigationDestination(
        icon: Icon(Icons.add_circle_outline),
        selectedIcon: Icon(Icons.add_circle),
        label: 'Add',
      ),
      NavigationDestination(
        icon: Icon(Icons.list_outlined),
        selectedIcon: Icon(Icons.list),
        label: 'Expenses',
      ),
      NavigationDestination(
        icon: Icon(Icons.person_outline),
        selectedIcon: Icon(Icons.person),
        label: 'Profile',
      ),
    ],
  ),
);
}

```

budget_notification_service.dart

```

import
'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';

class BudgetNotificationService {
  static final FlutterLocalNotificationsPlugin
_notifications =
FlutterLocalNotificationsPlugin();
  static bool _hasNotified = false;

```

```

  static final currencyFormat =
NumberFormat.currency(locale: 'en_IN',
symbol: '₹');

  static Future<void> initialize() async {
    const androidSettings =
AndroidInitializationSettings('@mipmap/ic_launcher');
    const initSettings =
InitializationSettings(android:
androidSettings);
    await _notifications.initialize(initSettings);
  }

  static Future<void>
showBudgetAlert(double percentage, double
remaining) async {
    if (_hasNotified) return;

    const androidDetails =

```

```

AndroidNotificationDetails(
  'budget_alert',
  'Budget Alerts',
  channelDescription: 'Alerts for budget
thresholds',
  importance: Importance.high,
  priority: Priority.high,
);

    await _notifications.show(
      0,
      'Budget Alert',
      'You have used
${percentage.toStringAsFixed(1)}% of your
budget.
${currencyFormat.format(remaining)}
remaining.',
      const NotificationDetails(android:
androidDetails),
    );

    _hasNotified = true;
  }

```

```

  static Future<void>
showOverBudgetAlert(double totalSpent,
double totalBudget) async {
    const androidDetails =
AndroidNotificationDetails(

```

```

    'over_budget',
    'Over Budget Alerts',
    channelDescription: 'Alerts when
spending exceeds the budget',
    importance: Importance.high,
    priority: Priority.high,
  );

  await _notifications.show(
    1,
    'Over Budget!',
    'You spent
    ${currencyFormat.format(totalSpent)} —
    more than your budget of
    ${currencyFormat.format(totalBudget)}!',
    const NotificationDetails(android:
    androidDetails),
  );

  _hasNotified = true;
}

/// Main budget check logic
static void checkBudget(BuildContext
context, double totalBudget, double
totalSpent) {
  double percentageUsed = (totalSpent /
totalBudget) * 100;
  double remaining = totalBudget -
totalSpent;

  if (totalSpent > totalBudget &&
!_hasNotified) {
    showOverBudgetAlert(totalSpent,
totalBudget);
    _showOverBudgetPopup(context,
totalSpent, totalBudget);
  } else if (percentageUsed >= 80 &&
!_hasNotified) {
    showBudgetAlert(percentageUsed,
remaining);
  } else if (percentageUsed < 80 &&
totalSpent <= totalBudget) {
    _hasNotified = false;
  }
}

///  Pop-up when over budget
static void
_showOverBudgetPopup(BuildContext

```

```

context, double totalSpent, double
totalBudget) {
  if (context.mounted) {
    showDialog(
      context: context,
      builder: (_) => AlertDialog(
        title: Text("Budget Exceeded!"),
        content: Text(
          "You've spent
          ${currencyFormat.format(totalSpent)} which
          is over your budget of
          ${currencyFormat.format(totalBudget)}.",
        ),
        actions: [
          TextButton(
            onPressed: () =>
Navigator.of(context).pop(),
            child: Text("Okay"),
          ),
        ],
      ),
    );
  }
}
}

```

budget_pie_chart.dart

```

import 'package:flutter/material.dart';
import 'package:fl_chart/fl_chart.dart';
import 'package:intl/intl.dart';
import 'expense_data.dart';

class BudgetPieChart extends StatefulWidget {
  final double totalBudget;
  final List<ExpenseData> expenses;

  const BudgetPieChart({
    Key? key,
    required this.totalBudget,
    required this.expenses,
  }) : super(key: key);

  @override
  State<BudgetPieChart> createState() =>
_BudgetPieChartState();
}

```

```

class _BudgetPieChartState extends
State<BudgetPieChart> {
  int touchedIndex = -1;
  final currencyFormat =
NumberFormat.currency(locale: 'en_IN',
symbol: '₹');

  @override
Widget build(BuildContext context) {
  return SingleChildScrollView(
    child: Column(
      children: [
        Container(
          height: 300,
          width: double.infinity,
          child: Card(
            elevation: 8,
            shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(24)),
            color: Colors.grey[50],
            child: Padding(
              padding: const EdgeInsets.all(20),

```

```

        child: PieChart(
          PieChartData(
            pieTouchData: PieTouchData(
              touchCallback: (FlTouchEvent
event, pieTouchResponse) {
                setState() {
                  if
(!event.isInterestedForInteractions ||
                    pieTouchResponse == null
||
                    pieTouchResponse.touchedSection == null) {
                    touchedIndex = -1;
                    return;
                  }
                  touchedIndex =
pieTouchResponse.touchedSection!.touchedS
ectionIndex;
                });
              },
            ),
            borderData: FlBorderData(show:
false),
            sectionsSpace: 3,
            centerSpaceRadius: 40,
            sections: showingSections(),

```

```

          ),
        ),
        ),
        ),
        SizedBox(height: 20),
        Container(
          padding: const EdgeInsets.all(20),
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius:
BorderRadius.circular(24),
            boxShadow: [
              BoxShadow(
                color:
Colors.grey.withOpacity(0.1),
                blurRadius: 10,
                spreadRadius: 2,
                offset: Offset(0, 4),
              ),
            ],
          ),
        ),
        child: Column(
          children: [
            Text(
              'Expense Breakdown',
              style: TextStyle(
                fontSize: 18,
                fontWeight: FontWeight.bold,
                color: Colors.grey[800],
              ),
            ),
            SizedBox(height: 16),
            ...widget.expenses.map((expense)
=> legendItem(expense)).toList(),
          ],
        ),
      ],
    );
  }

  List<PieChartSectionData>
showingSections() {
    return
List.generate(widget.expenses.length, (i) {

```

```

        final isTouched = i == touchedIndex;
        final double fontSize = isTouched ? 20 :
16;
        final double radius = isTouched ? 130 :
120;

        final double percentage =
(widget.expenses[i].amount /
widget.totalBudget) * 100;

        return PieChartSectionData(
            color:
_getColor(i).withOpacity(isTouched ? 1 :
0.9),
            value: widget.expenses[i].amount,
            title:
'$ {percentage.toStringAsFixed(1)}%',
            radius: radius,
            titleStyle: TextStyle(
                fontSize: fontSize,
                fontWeight: FontWeight.bold,
                color: const Color(0xffffffff),
                shadows: [Shadow(color:
Colors.black26, blurRadius: 2)],
            ),
        );
    });
}

Widget legendItem(ExpenseData expense) {
    final index =
widget.expenses.indexOf(expense);
    final isSelected = index == touchedIndex;

    return Container(
        margin: EdgeInsets.symmetric(vertical:
6),
        child: Row(
            children: [
                Container(
                    width: 20,
                    height: 20,
                    decoration: BoxDecoration(
                        shape: BoxShape.circle,
                        color: _getColor(index),
                        boxShadow: isSelected ? [
                            BoxShadow(

```

```

                                color:
_getColor(index).withOpacity(0.3),
                                blurRadius: 6,
                                spreadRadius: 2,
                            )
                        ] : [],
                    ),
                ),
                SizedBox(width: 12),
                Expanded(
                    child: Text(
                        expense.category,
                        style: TextStyle(
                            fontSize: 15,
                            fontWeight: isSelected ?
FontWeight.bold : FontWeight.normal,
                            color: Colors.grey[800],
                        ),
                    ),
                Text(
                    currencyFormat.format(expense.amount),
                    style: TextStyle(
                        fontSize: 15,
                        fontWeight: FontWeight.bold,
                    ),
                ),
            ],
        ),
    );
}

Color _getColor(int index) {
    List<Color> colors = [
        Color(0xFF2196F3), // Blue
        Color(0xFF4CAF50), // Green
        Color(0xFFFFA726), // Orange
        Color(0xFF9C27B0), // Purple
        Color(0xFFE91E63), // Pink
        Color(0xFF00BCD4), // Cyan
    ];
    return colors[index % colors.length];
}
}

```

expense_data.dart

```
class ExpenseData {
  final String category;
  final double amount;

  ExpenseData({required this.category,
required this.amount});
}
```

expense_model.dart

```
class Expense {
  final String id;
  final double amount;
  final String category;
  final DateTime date;
  final String description;

  Expense({required this.id, required
this.amount, required this.category, required
this.date, required this.description});

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'amount': amount,
      'category': category,
      'date': date.toIso8601String(),

      'description': description,
    };
  }

  factory Expense.fromJson(Map<String,
dynamic> json) {
    return Expense(
      id: json['id'] as String, // As seen in
      amount: (json['amount'] as
      int).toDouble(), // Amount appears as an
      category: json['category'] as String,
      date: DateTime.parse(json['date'] as
      String),
      description: json['description'] as String,
    );
  }
}
```

expense_service.dart

```
import
'package:firebase_auth/firebase_auth.dart';
import
'package:firebase_database/firebase_database.
dart';
import 'expense_model.dart';

class ExpenseService {
  final DatabaseReference _dbRef =
  FirebaseDatabase.instance.ref().child('expens
es');

  String getUserId() {
    final user =
    FirebaseAuth.instance.currentUser;
    return user != null ? user.uid : "";
  }

  void saveExpense(Expense expense) {
    String userId = getUserId();

    _dbRef.child(userId).child(expense.id).set(ex
    pense.toJson());
  }

  Stream<List<Expense>> getExpenses() {

    String userId = getUserId();
    return
    _dbRef.child(userId).onValue.map((event) {
      var data = event.snapshot.value as
      Map<dynamic, dynamic>?;
      if (data == null) return [];
      return data.values.map((e) =>
      Expense.fromJson(Map<String,
      dynamic>.from(e as Map))).toList();
    });
  }
}
```

firebase_options.dart

```
// File generated by FlutterFire CLI.
// ignore_for_file: type=lint
```

```

import
'package:firebase_core/firebase_core.dart'
show FirebaseOptions;
import 'package:flutter/foundation.dart'
show defaultTargetPlatform, kIsWeb,
TargetPlatform;

/// Default [FirebaseOptions] for use with
/// your Firebase apps.
///
/// Example:
/// ```dart
/// import 'firebase_options.dart';
/// // ...
/// await Firebase.initializeApp(
///   options:
DefaultFirebaseOptions.currentPlatform,
/// );
/// ```
class DefaultFirebaseOptions {
  static FirebaseOptions get currentPlatform {
    if (kIsWeb) {
      return web;
    }
    switch (defaultTargetPlatform) {
      case TargetPlatform.android:
        return android;
      case TargetPlatform.iOS:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been
configured for ios - '
          'you can reconfigure this by running the
FlutterFire CLI again.',
        );
      case TargetPlatform.macOS:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been
configured for macos - '
          'you can reconfigure this by running the
FlutterFire CLI again.',
        );
      case TargetPlatform.windows:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been
configured for windows - '
          'you can reconfigure this by running the
FlutterFire CLI again.',
        );
    }
  }
}

```

```

case TargetPlatform.linux:
  throw UnsupportedError(
    'DefaultFirebaseOptions have not been
configured for linux - '
    'you can reconfigure this by running the
FlutterFire CLI again.',
  );
default:
  throw UnsupportedError(
    'DefaultFirebaseOptions are not
supported for this platform.',
  );
}

static const FirebaseOptions web =
FirebaseOptions(
  apiKey:
'AIzaSyDCTQUrY9hE04yovlZzZ5s3hwusTa
a4Ps4',
  appId:
'1:11231607026:web:4c38361f36b739aa90b8
6b',
  messagingSenderId: '11231607026',
  projectId: 'cashsplash-f9660',
  authDomain: 'cashsplash-
f9660.firebaseio.com',
  databaseURL: 'https://cashsplash-f9660-
default-rtdb.firebaseio.com',
  storageBucket: 'cashsplash-
f9660.firebaseio.com',
  measurementId: 'G-MTXBC71VJN',
);

static const FirebaseOptions android =
FirebaseOptions(
  apiKey:
'AIzaSyDjd5PBjsIf8rHmqMfyRzsicNbPm1L
ktGU',
  appId:
'1:11231607026:android:3fa899c54858930b9
0b86b',
  messagingSenderId: '11231607026',
  projectId: 'cashsplash-f9660',
  databaseURL: 'https://cashsplash-f9660-
default-rtdb.firebaseio.com',
);

```

```

    storageBucket: 'cashsplash-
f9660.firebaseiostorage.app',
  );
}

home_screen.dart
import 'package:flutter/material.dart';
import
'package:firebase_auth/firebase_auth.dart';
import
'package:firebase_database/firebase_database.
dart';
import 'package:fl_chart/fl_chart.dart';
import 'package:intl/intl.dart';

import 'add_expense_screen.dart';
import 'budget_notification_service.dart';
import 'budget_pie_chart.dart';
import 'view_expenses_screen.dart';
import 'profile_settings_screen.dart';
import 'expense_data.dart';

class HomeScreen extends StatefulWidget {
  @override
  _HomeScreenState createState() =>
  _HomeScreenState();
}

class _HomeScreenState extends
State<HomeScreen> {
  double _totalBudget = 2000.0;
  double _remainingBudget = 2000.0;
  String _userName = "User";

  final currencyFormat =
NumberFormat.currency(locale: 'en_IN',
symbol: '₹');

  @override
  void initState() {
    super.initState();
    _fetchUserProfileAndBudget();
    BudgetNotificationService.initialize();
  }

  void _fetchUserProfileAndBudget() {
    final user =
FirebaseAuth.instance.currentUser;
    if (user != null) {
      final DatabaseReference userRef =
FirebaseDatabase.instance.ref('user_profiles/$

```

```

{user.uid}');
    userRef.once().then((DatabaseEvent
event) {
      final data = event.snapshot.value as
Map<dynamic, dynamic>;
      if (data != null) {
        setState() {
          _userName = data['name'] ??
_userName;
          _totalBudget =
double.tryParse(data['budgetGoal'].toString()
) ?? _totalBudget;
          _remainingBudget = _totalBudget;
        });
        calculateRemainingBudget();
      }
    });
  }
}

void calculateRemainingBudget() {
  FirebaseDatabase.instance.ref('expenses/${Fir
ebaseAuth.instance.currentUser?.uid}').onVal
ue.listen((event) {
    double totalSpent = 0;
    if (event.snapshot.value != null) {
      Map<dynamic, dynamic> expenses =
Map<dynamic,
dynamic>.from(event.snapshot.value as
Map);
      totalSpent = expenses.values.fold(0,
(prev, element) => prev +
double.parse(element['amount'].toString()));
    }

    setState() {
      _remainingBudget = _totalBudget -
totalSpent;

      BudgetNotificationService.checkBudget(cont
ext, _totalBudget, totalSpent);

    });
  });
}

void _navigateToScreen(BuildContext
context, Widget screen) {
  Navigator.push(
    context,

```

```
MaterialPageRoute(builder: (context) =>
screen),
);
}
```

```
Column(
  crossAxisAlignment:
CrossAxisAlignment.start,
  children: [
    Text(
      'Welcome back,',
      style: TextStyle(color:
Colors.white70, fontSize: 16),
    ),
    SizedBox(height: 4),
    Text(
      _userName,
      style: TextStyle(
        color: Colors.white,
        fontSize: 24,
        fontWeight:
FontWeight.bold,
      ),
    ),
  ],
),
IconButton(
  icon: Icon(Icons.person,
color: Colors.white),
  onPressed: () =>
_navigateToScreen(context,
ProfileSettingsScreen()),
),
],
),
),
),
),
```

```

        children: [
          Text(
            'Expense Overview',
            style: TextStyle(
              fontSize: 18,
              fontWeight:
FontWeight.bold,
            ),
          ),
          TextButton(
            onPressed: () =>
_navigateToScreen(context,
ViewExpensesScreen()),
            child: Text(
              'View All',
              style: TextStyle(
                color: Color(0xFF6B46C1),
              ),
            ),
          ),
        ],
      ),
      _buildExpensesSection(),
      SizedBox(height: 100),
    ],
  ),
),
],
),
);
}

```

```

Widget _buildBudgetCard() {
  return Card(
    elevation: 8,
    shadowColor: Colors.black26,
    shape:

```

```

RoundedRectangleBorder(borderRadius:
BorderRadius.circular(20)),
    child: Padding(

padding: EdgeInsets.all(20),
    child: Column(
      children: [
        Row(
          mainAxisAlignment:
MainAxisAlignment.spaceBetween,
          children: [
            Column(

```

```

      crossAxisAlignment:
CrossAxisAlignment.start,
      children: [
        Text(
          'Total Budget',
          style: TextStyle(
            fontSize: 14,
            color: Colors.grey[600],
          ),
        ),
        SizedBox(height: 4),
        Text(
          currencyFormat.format(_totalBudget),
          style: TextStyle(
            fontSize: 24,
            fontWeight: FontWeight.bold,
            color: Color(0xFF6B46C1),
          ),
        ),
        ],
      ),
      CircleAvatar(
        radius: 24,
        backgroundColor:
Color(0xFF6B46C1),
        child:
Icon(Icons.account_balance_wallet, color:
Colors.white),
      ),
    ],
  ),
  SizedBox(height: 20),
  Container(
    padding: EdgeInsets.all(15),
    decoration: BoxDecoration(
      color: _remainingBudget > 0 ?
Colors.green[50] : Colors.red[50],
      borderRadius:
BorderRadius.circular(12),
    ),
    child: Row(
      mainAxisAlignment:
MainAxisAlignment.spaceBetween,
      children: [
        Text(
          'Remaining Budget',
          style: TextStyle(

```

```

        color: _remainingBudget > 0 ?
Colors.green[700] : Colors.red[700],
      ),
    ),
    Text(
currencyFormat.format(_remainingBudget),
      style: TextStyle(
        fontWeight: FontWeight.bold,
        color: _remainingBudget > 0 ?
Colors.green[700] : Colors.red[700],
      ),
    ),
  ],
),
),
),
),
),
),
);
}

```

```

Widget _buildExpensesSection() {
  return StreamBuilder(
    stream:
FirebaseDatabase.instance.ref('expenses/${Fir
ebaseAuth.instance.currentUser?.uid}').onVal
ue,
    builder: (context,
AsyncSnapshot<DatabaseEvent> snapshot) {
      if (snapshot.connectionState ==
ConnectionState.waiting) {
        return Center(child:
CircularProgressIndicator(
          color: Color(0xFF6B46C1),
        ));
      }

```

```

    if (snapshot.hasError) {
      return _buildErrorCard();
    }

    if (!snapshot.hasData ||
snapshot.data!.snapshot.value == null) {
      return _buildEmptyState();
    }

```

```

    Map<dynamic, dynamic> expensesData
= Map<dynamic,
dynamic>.from(snapshot.data!.snapshot.value

```

```

as Map);
    List<ExpenseData> expenses =
expensesData.entries.map((e) {
      return ExpenseData(
        category: e.value['category'],
        amount:
double.tryParse(e.value['amount'].toString())
?? 0.0,
      );
    }).toList();

    return Container(
      margin: EdgeInsets.only(bottom: 20),
      child: BudgetPieChart(
        totalBudget: _totalBudget,
        expenses: expenses,
      ),
    );
  },
);
}

```

```

Widget _buildErrorCard() {
  return Container(
    height: 200,
    margin: EdgeInsets.symmetric(vertical:
10),
    child: Card(
      elevation: 8,
      shadowColor: Colors.black26,
      shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(20)),
      color: Colors.pink[50],
      child: Column(
        mainAxisAlignment:

```

```

MainAxisAlignment.center,
        children: [
          Icon(Icons.error_outline,
            size: 48,
            color: Colors.red[400]
          ),
          SizedBox(height: 16),
          Text(
            'Error loading expenses data',
            style: TextStyle(
              fontSize: 16,
              color: Colors.red[400],
              fontWeight: FontWeight.w500
            ),

```

```

    ),
  ],
),
);
}

Widget _buildEmptyState() {
  return Container(
    height: 200,
    margin: EdgeInsets.symmetric(vertical:
10),
    child: Card(
      elevation: 8,
      shadowColor: Colors.black26,
      shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(20)),
      color: Colors.pink[50],
      child: Column(
        mainAxisAlignment:
MainAxisAlignment.center,
        children: [

Icon(Icons.account_balance_wallet_outlined,
      size: 48,
      color: Color(0xFF6B46C1)
    ),
    SizedBox(height: 16),
    Text(
      'No expenses recorded yet',
      style: TextStyle(
        fontSize: 16,
        color: Color(0xFF6B46C1),
        fontWeight: FontWeight.w500
      ),
    ),
  ],
),
],
),
);
}
}

```

login_screen.dart

```

import 'package:flutter/material.dart';
import
'package:firebase_auth/firebase_auth.dart';
import 'package:project/home_screen.dart';

```

```

import 'signup_screen.dart';

class LoginScreen extends StatefulWidget {
  @override
  _LoginScreenState createState() =>
_LoginScreenState();
}

class _LoginScreenState extends
State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();
  final _emailController =
TextEditingController();
  final _passwordController =
TextEditingController();
  bool _isLoading = false;
  bool _obscurePassword = true;

  Future<void> signIn() async {
    if (!_formKey.currentState!.validate())
return;

    setState(() => _isLoading = true);
    try {
      await
FirebaseAuth.instance.signInWithEmailAndP
assword(
        email: _emailController.text.trim(),
        password: _passwordController.text,
      );
      Navigator.of(context).pushReplacement(
        MaterialPageRoute(builder: (context)
=> HomeScreen())
      );

    } on FirebaseAuthException catch (e) {

ScaffoldMessenger.of(context).showSnackBa
r(SnackBar(
      content: Text(e.message ??
'Authentication failed'),
      behavior: SnackBarBehavior.floating,
      backgroundColor:
Theme.of(context).colorScheme.error,
    ));
    } finally {
      if (mounted) setState(() => _isLoading =
false);
    }
  }
}

```

```

    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Center(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(24),
            child: Column(
              mainAxisAlignment:
MainAxisAlignment.center,
              crossAxisAlignment:
CrossAxisAlignment.stretch,
              children: [
                Icon(
                  Icons.account_balance_wallet,
                  size: 64,
                  color:
Theme.of(context).colorScheme.primary,
                ),
                const SizedBox(height: 24),
                Text(
                  'Welcome Back',
                  style:
Theme.of(context).textTheme.headlineMedium,
                  textAlign: TextAlign.center,
                ),
                const SizedBox(height: 8),
                Text(
                  'Sign in to continue',
                  style:
Theme.of(context).textTheme.bodyLarge?.copyWith(
                    color:

Theme.of(context).colorScheme.onSurfaceVariant,
                  ),
                  textAlign: TextAlign.center,
                ),
                const SizedBox(height: 32),
                Form(
                  key: _formKey,
                  child: Column(
                    children: [
                      TextFormField(

```

```

                        controller: _emailController,
                        keyboardType:
TextInputType.emailAddress,
                        decoration: const
InputDecoration(
                          labelText: 'Email',
                          prefixIcon:
Icon(Icons.email_outlined),
                        ),
                        validator: (value) {
                          if (value?.isEmpty ?? true)
return 'Please enter your email';
                          if (!value!.contains('@'))
return 'Please enter a valid email';
                          return null;
                        },
                      ),
                      const SizedBox(height: 16),
                      TextFormField(
                        controller:
_passwordController,
                        obscureText:
_obscurePassword,
                        decoration: InputDecoration(
                          labelText: 'Password',
                          prefixIcon: const
Icon(Icons.lock_outline),
                          suffixIcon: IconButton(
                            icon:
Icon(_obscurePassword
                              ? Icons.visibility_outlined
                              :
Icons.visibility_off_outlined
                            ),
                            onPressed: () => setState(()
=> _obscurePassword = !_obscurePassword),
                          ),
                        ),
                        validator: (value) =>
                          value?.isEmpty ?? true ?

'Please enter your password' : null,
                      ),

```

```

    ],
  ),
),
const SizedBox(height: 24),
FilledButton(
  onPressed: _isLoading ? null :
signIn,
  child: _isLoading
    ? const SizedBox(
      height: 20,
      width: 20,
      child: CircularProgressIndicator(
        strokeWidth: 2,
        color: Colors.white,
      ),
    )
    : const Text('Sign In'),
),
const SizedBox(height: 16),
TextButton(
  onPressed: () => Navigator.push(
    context,
    MaterialPageRoute(builder:
(context) => SignupScreen()),
  ),
  child: Text.rich(
    TextSpan(
      children: [
        const TextSpan(text: "Don't
have an account? "),
        TextSpan(
          text: 'Sign up',
          style: TextStyle(
            color:
Theme.of(context).colorScheme.primary,
fontWeight:
FontWeight.bold,
          ),
        ),
      ],
    ),
  ),
),
),
],
),
),
),
);
}
}

```

profile_setting_screen.dart

```

import 'dart:typed_data';
import 'package:flutter/material.dart';
import
'package:firebase_auth/firebase_auth.dart';
import
'package:firebase_database/firebase_database.
dart';
import
'package:image_picker/image_picker.dart';
import 'dart:convert';
import 'login_screen.dart';

class ProfileSettingsScreen extends
StatefulWidget {
  @override
  _ProfileSettingsScreenState createState() =>
_ProfileSettingsScreenState();
}

class _ProfileSettingsScreenState extends
State<ProfileSettingsScreen> {
  final TextEditingController _nameController
= TextEditingController();
  final TextEditingController
_budgetController = TextEditingController();
  final DatabaseReference dbRef =
FirebaseDatabase.instance.ref();
  bool _isLoading = true;
  String _imageBase64 = "";
  final picker = ImagePicker();

  @override
  void initState() {
    super.initState();
    fetchUserProfile();
  }

  void fetchUserProfile() {
    final user =
FirebaseAuth.instance.currentUser;
    if (user != null) {

dbRef.child('user_profiles').child(user.uid).on
ce().then((snapshot) {
      if (snapshot.snapshot.value != null) {
        Map<String, dynamic> data =
Map<String,
dynamic>.from(snapshot.snapshot.value as
Map);

```

```

        _nameController.text = data['name'] ??
";
        _budgetController.text =
data['budgetGoal'] ?? ";

        _imageBase64 = data['image'] ?? ";
    }
    setState(() => _isLoading = false);
  });
} else {
  setState(() => _isLoading = false);
}
}

Future<void> pickImageAndEncode() async
{
  final pickedFile = await
picker.pickImage(source:
ImageSource.gallery);
  if (pickedFile != null) {
    Uint8List bytes = await
pickedFile.readAsBytes();
    setState(() => _imageBase64 =
base64Encode(bytes));
  }
}

void saveProfileSettings() {
  final user =
FirebaseAuth.instance.currentUser;
  if (user != null) {

dbRef.child('user_profiles').child(user.uid).update({
  'name': _nameController.text,

```

```

  'budgetGoal': _budgetController.text,
  'image': _imageBase64,
}).then((_) {

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: const Text('Profile settings
saved'),
    behavior: SnackBarBehavior.floating,
    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),

```

```

    ));
  }).catchError((error) {

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Text('Failed to save: $error'),
    backgroundColor:
Theme.of(context).colorScheme.error,
    behavior: SnackBarBehavior.floating,

    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
  ));
});
}

void logout() async {
  await FirebaseAuth.instance.signOut();
  Navigator.of(context).pushReplacement(
    MaterialPageRoute(builder: (context) =>
LoginScreen()));
}

@override
Widget build(BuildContext context) {
  if (_isLoading) {
    return Scaffold(
      body: Center(child:
CircularProgressIndicator()),
    );
  }

  return Scaffold(

```

```

appBar: AppBar(
  title: const Text('Profile'),
  centerTitle: true,
  actions: [
    IconButton(
      icon: const Icon(Icons.logout),
      onPressed: () => showDialog(
        context: context,
        builder: (context) => AlertDialog(
          title: const Text('Logout'),
          content: const Text('Are you sure
you want to logout?'),

```

```

actions: [
  TextButton(
    child: const Text('Cancel'),
    onPressed: () =>
Navigator.pop(context),
  ),
  FilledButton(
    child: const Text('Logout'),
    onPressed: () {
      Navigator.pop(context);
      logout();
    },
  ),
],
),
],
),
),
],
),
body: SingleChildScrollView(
  child: Column(
    children: [
      Container(
        width: double.infinity,
        color:
Theme.of(context).colorScheme.surfaceVari-
ant,
        child: Column(
          children: [
            const SizedBox(height: 20),
            Stack(
              children: [
                InkWell(
                  onTap: pickImageAndEncode,
                  child: Hero(
                    tag: 'profile-image',
                    child: Container(
                      width: 120,
                      height: 120,

```

```

decoration: BoxDecoration(
  shape: BoxShape.circle,
  border: Border.all(
    color:
Theme.of(context).colorScheme.primary,
    width: 3,
  ),
  image: DecorationImage(
    fit: BoxFit.cover,

```

```

image:
  _imageBase64.isEmpty
    ? const
NetworkImage('https://via.placeholder.com/150')
    :
MemoryImage(base64Decode(_imageBase64
)) as ImageProvider,
),
),
),
),
),
Positioned(
  bottom: 0,
  right: 0,
  child: Container(
    padding: const
EdgeInsets.all(4),
    decoration: BoxDecoration(
      color:
Theme.of(context).colorScheme.primary,
      shape: BoxShape.circle,
    ),
    child: Icon(
      Icons.edit,
      size: 20,
      color:
Theme.of(context).colorScheme.onPrimary,
    ),
  ),
),
const SizedBox(height: 20),
],
),
),
Padding(
  padding: const EdgeInsets.all(16),

```

```

child: Column(
  crossAxisAlignment:
CrossAxisAlignment.stretch,
  children: [
    Card(
      child: Padding(

```

```

padding: const
EdgeInsets.all(16),
child: Column(
  crossAxisAlignment:
CrossAxisAlignment.start,
  children: [
    Text(
      'Personal Information',
      style:
Theme.of(context).textTheme.titleMedium,
    ),
    const SizedBox(height: 16),
    TextFormField(
      controller: _nameController,
      decoration: const
InputDecoration(
  labelText: 'Name',
  prefixIcon:
Icon(Icons.person_outline),
),
),
    const SizedBox(height: 16),
    TextFormField(
      controller:
_budgetController,
      decoration: const
InputDecoration(
  labelText: 'Monthly
Budget Goal',
  prefixIcon:
Icon(Icons.attach_money),
),
      keyboardType:
TextInputType.numberWithOptions(decimal:
true),
    ),
  ],
),
),
const SizedBox(height: 20),
FilledButton.icon(
  onPressed: saveProfileSettings,
  icon: const Icon(Icons.save),
  label: const Text('Save
Changes'),
),
),
),

```

```

),
],
),
);
}
}
signup_screen.dart
import 'package:flutter/material.dart';
import
'package:firebase_auth/firebase_auth.dart';
import
'package:firebase_database/firebase_database.
dart';
import 'home_screen.dart';

class SignupScreen extends StatefulWidget {
  @override
  _SignupScreenState createState() =>
_SignupScreenState();
}

class _SignupScreenState extends
State<SignupScreen> with
SingleTickerProviderStateMixin {
  final _formKey = GlobalKey<FormState>();
  final _emailController =
TextEditingController();
  final _passwordController =
TextEditingController();
  final _nameController =
TextEditingController();
  final _budgetController =
TextEditingController();
  late AnimationController
_animationController;
  late Animation<double> _fadeAnimation;
  bool _isLoading = false;
  bool _obscurePassword = true;

  @override

  void initState() {
    super.initState();
    _animationController =
AnimationController(
      vsync: this,

```

```

        duration: const Duration(milliseconds:
800),
    );
    _fadeAnimation = Tween<double>(begin:
0.0, end: 1.0).animate(
        CurvedAnimation(parent:
_animationController, curve:
Curves.easeOut),
    );
    _animationController.forward();
}

@override
void dispose() {
    _animationController.dispose();
    super.dispose();
}

Future<void> signUp() async {
    if (!_formKey.currentState!.validate())
return;

    setState(() => _isLoading = true);
    try {
        UserCredential userCredential = await
FirebaseAuth.instance
            .createUserWithEmailAndPassword(
                email: _emailController.text.trim(),
                password: _passwordController.text);

        await FirebaseDatabase.instance

.ref('user_profiles/${userCredential.user!.uid}
')
        .set({
            'name': _nameController.text,
            'budgetGoal': _budgetController.text,
            'createdAt':
DateTime.now().toIso8601String(),
        });

        if (mounted) {
            Navigator.of(context).pushReplacement(
                PageRouteBuilder(
                    pageBuilder: (context, animation,
secondaryAnimation) => HomeScreen(),
                    transitionsBuilder: (context,
animation, secondaryAnimation, child) {
                        return FadeTransition(opacity:
animation, child: child);

```

```

    },
    ),
    );
}
} on FirebaseAuthException catch (e) {

ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
        content: Text(e.message ?? 'Sign up
failed'),
        behavior: SnackBarBehavior.floating,
        backgroundColor:
Theme.of(context).colorScheme.error,
        shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
    ));
} finally {
    if (mounted) setState(() => _isLoading =
false);
}
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        body: Container(
            decoration: BoxDecoration(
                gradient: LinearGradient(
                    begin: Alignment.topLeft,
                    end: Alignment.bottomRight,
                    colors: [

Theme.of(context).colorScheme.primaryCont
ainer.withOpacity(0.3),

Theme.of(context).colorScheme.surface,
                ],
            ),
        ),
        child: SafeArea(

child: Stack(
            children: [
                SingleChildScrollView(
                    padding: const EdgeInsets.all(24),
                    child: FadeTransition(
                        opacity: _fadeAnimation,
                        child: Column(

```

```

        crossAxisAlignment:
CrossAxisAlignment.stretch,
        children: [
            const SizedBox(height: 40),
            Icon(
Icons.account_balance_wallet_outlined,
                size: 64,
                color:
Theme.of(context).colorScheme.primary,
            ),
            const SizedBox(height: 24),
            Text(
                'Join ExpenseTracker',
                style:
Theme.of(context).textTheme.headlineMediu
m?.copyWith(
                    fontWeight:
FontWeight.bold,
                ),
                textAlign: TextAlign.center,
            ),
            const SizedBox(height: 8),
            Text(
                'Take control of your finances',
                style:
Theme.of(context).textTheme.bodyLarge?.co
pyWith(
                    color:
Theme.of(context).colorScheme.onSurfaceVa
riant,
                ),
                textAlign: TextAlign.center,
            ),
            const SizedBox(height: 40),
            Card(
                elevation: 0,
                color:
Theme.of(context).colorScheme.surface.with
Opacity(0.8),
                child: Padding(
                    padding: const
EdgeInsets.all(20),

                    child: Form(
                        key: _formKey,
                        child: Column(
                            children: [
                                _buildTextField(
                                    controller:
_nameController,

```

```

                                label: 'Full Name',
                                icon:
Icons.person_outline,
                                validator: (v) =>
v?.isEmpty ?? true ? 'Required' : null,
                                ),
                                const SizedBox(height:
16),
                                _buildTextField(
                                    controller:
_emailController,
                                    label: 'Email',
                                    icon:
Icons.email_outlined,
                                    keyboardType:
TextInputType.emailAddress,
                                    validator: (v) {
                                        if (v?.isEmpty ?? true)
return 'Required';
                                        if (!v!.contains('@'))
return 'Invalid email';
                                        return null;
                                    },
                                ),
                                const SizedBox(height:
16),
                                _buildTextField(
                                    controller:
_passwordController,
                                    label: 'Password',
                                    icon: Icons.lock_outline,
                                    obscureText:
_obscurePassword,
                                    validator: (v) {
                                        if (v?.isEmpty ?? true)
return 'Required';
                                        if (v!.length < 6) return
'Min 6 characters';
                                        return null;
                                    },
                                ),
                                suffixIcon: IconButton(
                                    icon:
Icon(_obscurePassword
?
Icons.visibility_outlined
:
Icons.visibility_off_outlined),
                                    onPressed: () =>
setState(() =>
_obscurePassword = !_obscurePassword),

```

```

        ),
      ),
      const SizedBox(height:
16),
        _buildTextField(
          controller:
            _budgetController,
            label: 'Monthly Budget
Goal',
            icon:
Icons.savings_outlined,
            keyboardType:
TextInputType.number,
            prefixText: '\$',
            validator: (v) =>
v?.isEmpty ?? true ? 'Required' : null,
        ),
      ],
    ),
  ),
),
const SizedBox(height: 24),
FilledButton(
  onPressed: _isLoading ? null :
signUp,
  style: FilledButton.styleFrom(
    padding: const
EdgeInsets.all(16),
    shape:
RoundedRectangleBorder(
      borderRadius:
BorderRadius.circular(12),
    ),
  ),
  child: _isLoading
    ? const SizedBox(
      height: 20,
      width: 20,

      child: CircularProgressIndicator(
        strokeWidth: 2,
        color: Colors.white,
      ),
    )
    : const Text('Create
Account'),
  ),
],

```

```

    ),
  ),
),
SafeArea(
  child: IconButton(
    icon: const
Icon(Icons.arrow_back),
    onPressed: () =>
Navigator.pop(context),
  ),
),
],
),
),
),
);
}

Widget _buildTextField({
  required TextEditingController controller,
  required String label,
  required IconData icon,
  String? Function(String?)? validator,
  TextInputType? keyboardType,
  bool obscureText = false,
  Widget? suffixIcon,
  String? prefixText,
}) {
  return TextFormField(
    controller: controller,
    obscureText: obscureText,
    keyboardType: keyboardType,
    style:
Theme.of(context).textTheme.bodyLarge,
    decoration: InputDecoration(
      labelText: label,

      prefixIcon: Icon(icon),
      prefixText: prefixText,
      suffixIcon: suffixIcon,
      floatingLabelBehavior:
FloatingLabelBehavior.auto,
      border: OutlineInputBorder(
        borderRadius:
BorderRadius.circular(12),
      ),
    ),
    validator: validator,

```

```
);
}
}
```

View_expense_screen.dart

```
import
'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import
'package:firebase_database/firebase_database.
dart';
import 'dart:convert';
import 'package:intl/intl.dart';
import 'dart:typed_data'; // Needed for
Uint8List
```

```
class ViewExpensesScreen extends
StatefulWidget {
  @override
  _ViewExpensesScreenState createState() =>
  _ViewExpensesScreenState();
}
```

```
class _ViewExpensesScreenState extends
State<ViewExpensesScreen> {
  List<Map<String, dynamic>> expenses = [];
  final currencyFormatter =
NumberFormat.currency(locale: 'en_IN',
symbol: '₹');
  Map<String, String> expenseIds = {};
```

```
@override
void initState() {
  super.initState();
  _loadExpenses();
}
```

```
void _loadExpenses() {
  DatabaseReference dbRef =
FirebaseDatabase.instance.ref("expenses/${Fi
rebaseAuth.instance.currentUser?.uid}");
  dbRef.onValue.listen((DatabaseEvent
event) {
    final data = event.snapshot.value;
    if (data != null && data is Map<dynamic,
dynamic>) {
      setState() {
```

```
expenseIds.clear();
expenses.clear();
```

```
data.forEach((key, value) {
  final expenseData = Map<String,
dynamic>.from(value as Map);
  expenses.add(expenseData);
  expenseIds[expenseData['date']] =
key.toString();
});

expenses.sort((a, b) =>
DateTime.parse(b['date']).compareTo(DateTi
me.parse(a['date'])));
});
} else {
  setState() {
    expenses = [];
    expenseIds.clear();
  });
}, onError: (error) {
  print("Failed to load expenses: $error");
  setState() {
    expenses = [];
    expenseIds.clear();
  });
});
}
```

```
Future<void> _deleteExpense(int index)
async {
  final expense = expenses[index];
  final expenseId =
expenseIds[expense['date']];
```

```
if (expenseId == null) return;
```

```
try {
  await FirebaseDatabase.instance
.ref("expenses/${FirebaseAuth.instance.curre
ntUser?.uid}/$expenseId")
.remove();
```

```
ScaffoldMessenger.of(context).showSnackBa
r(
```

```

SnackBar(
  content: Row(
    children: [
      Icon(Icons.check_circle, color:
Colors.white),
      SizedBox(width: 8),
      Text('Expense deleted successfully'),
    ],
  ),
  backgroundColor: Colors.green,
  behavior: SnackBarBehavior.floating,
  shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
);
} catch (error) {

```

```

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Text('Failed to delete expense:
$error'),
    backgroundColor: Colors.red,
    behavior: SnackBarBehavior.floating,
    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
  ),
);
}
}

```

```

Future<bool> _confirmDelete() async {
  final result = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      title: Text('Delete Expense'),

```

```

      content: Text('Are you sure you want to
delete this expense?'),
      actions: [
        TextButton(
          onPressed: () =>
Navigator.of(context).pop(false),
          child: Text('Cancel'),
        ),
        TextButton(
          onPressed: () =>
Navigator.of(context).pop(true),

```

```

        style: TextButton.styleFrom(
          foregroundColor: Colors.red,
        ),
        child: Text('Delete'),
      ),
    ],
  ),
);
return result ?? false;
}

```

```

Color _getCategoryColor(String category) {
  final colors = {
    'Food': Colors.orange,
    'Transport': Colors.blue,
    'Shopping': Colors.pink,
    'Entertainment': Colors.purple,
  };
  return colors[category] ?? Colors.teal;
}

```

```

void _showReceipt(String? base64Image) {
  if (base64Image == null ||
base64Image.isEmpty) {

```

```

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: const Text("No receipt image
available"),
    behavior: SnackBarBehavior.floating,
    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
  ),

```

```

);
return;
}

try {
  Uint8List imageBytes =
base64Decode(base64Image); // Ensures
proper decoding
  showDialog(
    context: context,
    builder: (context) => Dialog(
      child: Column(

```

```

        mainAxisSize: MainAxisSize.min,
        children: [
          AppBar(
            title: const Text("Receipt"),
            centerTitle: true,
            automaticallyImplyLeading: false,
            actions: [
              IconButton(
                icon: const Icon(Icons.close),
                onPressed: () =>
Navigator.of(context).pop(),
              ),
            ],
          ),
          Flexible(
            child: SingleChildScrollView(
              child:
Image.memory(imageBytes), // Displays
decoded image
            ),
          ),
        ],
      ),
    );
  } catch (e) {

ScaffoldMessenger.of(context).showSnackBar(
  r(
    SnackBar(
      content: const Text("Failed to load
receipt image"),
      behavior: SnackBarBehavior.floating,
      shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),

    ),
  );
}
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Expenses'),
      centerTitle: true,
      elevation: 0,

```

```

        scrolledUnderElevation: 1,
      ),
      body: expenses.isEmpty
        ? Center(
            child: Column(
              mainAxisAlignment:
MainAxisAlignment.center,
              children: [
                Icon(Icons.receipt_long_outlined,
                  size: 64,
                  color:
Theme.of(context).colorScheme.secondary),
                const SizedBox(height: 16),
                Text(
                  'No expenses yet',
                  style:
Theme.of(context).textTheme.titleLarge,
                ),
              ],
            ),
          )
        : CustomScrollView(
            slivers: [
              SliverPadding(
                padding: const EdgeInsets.all(16),
                sliver: SliverList(
                  delegate:
SliverChildBuilderDelegate(
                    (context, index) {
                      var expense = expenses[index];
                      String formattedDate =
DateFormat('MMM d,
y').format(DateTime.parse(expense['date']));

                      return Dismissible(
                        key: Key(expense['date']),
                        direction:
DismissDirection.endToStart,
                        confirmDismiss: (direction) =>
_confirmDelete(),
                        onDismissed: (direction) =>
_deleteExpense(index),
                        background: Container(
                          alignment:
Alignment.centerRight,
                          padding: EdgeInsets.only(right:
20),
                          decoration: BoxDecoration(
                            color: Colors.red,
                            borderRadius:

```

```

BorderRadius.circular(12),
    ),
    child: Icon(
      Icons.delete,
      color: Colors.white,
    ),
  ),
  child: Card(
    elevation: 2,
    margin: const
EdgeInsets.only(bottom: 12),
    child: InkWell(
      onTap: () =>
_showReceipt(expense['receiptImage']),
      borderRadius:
BorderRadius.circular(12),
      child: Padding(
        padding: const
EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment:
CrossAxisAlignment.start,
          children: [
            Row(
              mainAxisAlignment:
MainAxisAlignment.spaceBetween,
              children: [
                Expanded(
                  child: Text(
                    expense['category'],
                    style:
Theme.of(context).textTheme.titleMedium,
                  ),
                ),
                ),
                Container(
                  padding: const
EdgeInsets.symmetric(horizontal: 12, vertical:
6),
                  decoration:
BoxDecoration(
                    color:
_getCategoryColor(expense['category']).with
Opacity(0.1),
                    borderRadius:
BorderRadius.circular(20),
                  ),
                  child: Text(
                    currencyFormatter.format(double.parse(expe
nse['amount']).toString()),

```

```

style: TextStyle(
  color:
_getCategoryColor(expense['category']),
  fontWeight:
FontWeight.bold,
),
),
),
],
),
const SizedBox(height: 8),
Row(
  mainAxisAlignment:
MainAxisAlignment.spaceBetween,
  children: [
    Text(
      formattedDate,
      style:
Theme.of(context).textTheme.bodyMedium?.
copyWith(
        color:
Theme.of(context).colorScheme.onSurfaceVa
riant,
      ),
    ),
    if
(expense['receiptImage'] != null)
    Icon(
      Icons.receipt_outlined,
      size: 20,
      color:
Theme.of(context).colorScheme.primary,
    ),
  ],
),
],
),
),
),
),
),
);
},
childCount: expenses.length,
),
),
),
],
),
);

```

```

}
}

```

profile_settings_screen.dart

```

import
'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import
'package:firebase_database/firebase_database.
dart';
import 'dart:convert';
import 'package:intl/intl.dart';
import 'dart:typed_data'; // Needed for
Uint8List

```

```

class ViewExpensesScreen extends
StatefulWidget {
  @override
  _ViewExpensesScreenState createState() =>
  _ViewExpensesScreenState();
}

```

```

class _ViewExpensesScreenState extends
State<ViewExpensesScreen> {
  List<Map<String, dynamic>> expenses = [];
  final currencyFormatter =
NumberFormat.currency(locale: 'en_IN',

```

```

symbol: '₹');
  Map<String, String> expenseIds = {};

```

```

  @override
  void initState() {
    super.initState();
    _loadExpenses();
  }

```

```

  void _loadExpenses() {
    DatabaseReference dbRef =
FirebaseDatabase.instance.ref("expenses/${Fi
rebaseAuth.instance.currentUser?.uid}");
    dbRef.onValue.listen((DatabaseEvent
event) {
      final data = event.snapshot.value;
      if (data != null && data is Map<dynamic,
dynamic>) {
        setState() {
          expenseIds.clear();
          expenses.clear();

```

```

          data.forEach((key, value) {
            final expenseData = Map<String,
dynamic>.from(value as Map);
            expenses.add(expenseData);
            expenseIds[expenseData['date']] =
key.toString();
          });

```

```

          expenses.sort((a, b) =>
DateTime.parse(b['date']).compareTo(DateTi
me.parse(a['date'])));
        });
      } else {
        setState() {
          expenses = [];
          expenseIds.clear();
        });
      }
    }, onError: (error) {
      print("Failed to load expenses: $error");
      setState() {
        expenses = [];
        expenseIds.clear();
      });
    });
  }

```

```

  Future<void> _deleteExpense(int index)
  async {
    final expense = expenses[index];
    final expenseId =
expenseIds[expense['date']];

```

```

    if (expenseId == null) return;

```

```

    try {
      await FirebaseDatabase.instance

```

```

.ref("expenses/${FirebaseAuth.instance.curre
ntUser?.uid}/${expenseId}")
      .remove();

```

```

ScaffoldMessenger.of(context).showSnackBar(

```

```

  SnackBar(
    content: Row(
      children: [
        Icon(Icons.check_circle, color:

```

```

Colors.white),
    SizedBox(width: 8),
    Text('Expense deleted successfully'),
  ],
),
backgroundColor: Colors.green,
behavior: SnackBarBehavior.floating,
shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
),
);
} catch (error) {

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Text('Failed to delete expense:
$error'),
    backgroundColor: Colors.red,
    behavior: SnackBarBehavior.floating,
    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
  ),
);
}

Future<bool> _confirmDelete() async {
  final result = await showDialog<bool>(<
    context: context,
    builder: (context) => AlertDialog(
      title: Text('Delete Expense'),
      content: Text('Are you sure you want to
delete this expense?'),
      actions: [
        TextButton(
          onPressed: () =>
Navigator.of(context).pop(false),
          child: Text('Cancel'),
        ),
        TextButton(
          onPressed: () =>
Navigator.of(context).pop(true),
          style: TextButton.styleFrom(
            foregroundColor: Colors.red,
          ),
          child: Text('Delete'),
        ),
      ],
    ),
  );
}

```

```

),
);
return result ?? false;
}

Color _getCategoryColor(String category) {
  final colors = {
    'Food': Colors.orange,
    'Transport': Colors.blue,
    'Shopping': Colors.pink,
    'Entertainment': Colors.purple,
  };
  return colors[category] ?? Colors.teal;
}

void _showReceipt(String? base64Image) {
  if (base64Image == null ||
base64Image.isEmpty) {

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: const Text("No receipt image
available"),
    behavior: SnackBarBehavior.floating,
    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
  ),
);
return;
}

try {
  Uint8List imageBytes =
base64Decode(base64Image); // Ensures
proper decoding
  showDialog(
    context: context,
    builder: (context) => Dialog(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          AppBar(
            title: const Text("Receipt"),
            centerTitle: true,
            automaticallyImplyLeading: false,
            actions: [
              IconButton(
                icon: const Icon(Icons.close),
                onPressed: () =>

```

```

Navigator.of(context).pop(),
      ),
    ],
  ),
  Flexible(
    child: SingleChildScrollView(
      child:
Image.memory(imageBytes), // Displays
decoded image
    ),
  ),
],
),
);
} catch (e) {

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: const Text("Failed to load
receipt image"),
    behavior: SnackBarBehavior.floating,
    shape:
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
  ),
);
}
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Expenses'),
      centerTitle: true,
      elevation: 0,
      scrolledUnderElevation: 1,
    ),
    body: expenses.isEmpty
      ? Center(
        child: Column(
          mainAxisAlignment:
MainAxisAlignment.center,
          children: [
            Icon(Icons.receipt_long_outlined,
              size: 64,
              color:

```

```

Theme.of(context).colorScheme.secondary),
      const SizedBox(height: 16),
      Text(
        'No expenses yet',
        style:
Theme.of(context).textTheme.titleLarge,
      ),
    ],
  ),
)
  : CustomScrollView(
    slivers: [
      SliverPadding(

padding: const EdgeInsets.all(16),
      sliver: SliverList(
        delegate:
SliverChildBuilderDelegate(
          (context, index) {
            var expense = expenses[index];
            String formattedDate =
DateFormat('MMM d,
y').format(DateTime.parse(expense['date']));

            return Dismissible(
              key: Key(expense['date']),
              direction:
DismissDirection.endToStart,
              confirmDismiss: (direction) =>
                _confirmDelete(),
              onDismissed: (direction) =>
                _deleteExpense(index),
              background: Container(
                alignment:
Alignment.centerRight,
                padding: EdgeInsets.only(right:
20),
                decoration: BoxDecoration(
                  color: Colors.red,
                  borderRadius:
BorderRadius.circular(12),
                ),
                child: Icon(
                  Icons.delete,
                  color: Colors.white,
                ),
              ),
              child: Card(
                elevation: 2,

```

```

        margin: const
EdgeInsets.only(bottom: 12),
        child: InkWell(
          onTap: () =>
            _showReceipt(expense['receiptImage']),

        borderRadius:

BorderRadius.circular(12),
        child: Padding(
          padding: const
EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
          Row(
            mainAxisAlignment:

MainAxisAlignment.spaceBetween,
        children: [
          Expanded(
            child: Text(
              expense['category'],
              style:
Theme.of(context).textTheme.titleMedium,
            ),
          ),
          Container(
            padding: const
EdgeInsets.symmetric(horizontal: 12, vertical:
6),
            decoration:
BoxDecoration(
              color:
_getCategoryColor(expense['category']).with
Opacity(0.1),
              borderRadius:
BorderRadius.circular(20),
            ),
            child: Text(

currencyFormatter.format(double.parse(expe
nse['amount']).toString()),
            style: TextStyle(
              color:
_getCategoryColor(expense['category']),
              fontWeight:

```

```

FontWeight.bold,

),

),

),

],
),
const SizedBox(height: 8),
Row(
  mainAxisAlignment:
MainAxisAlignment.spaceBetween,
  children: [
    Text(
      formattedDate,
      style:
Theme.of(context).textTheme.bodyMedium?.
copyWith(
        color:
Theme.of(context).colorScheme.onSurfaceVa
riant,
      ),
    ),
    if
(expense['receiptImage'] != null)
      Icon(
        Icons.receipt_outlined,
        size: 20,
        color:
Theme.of(context).colorScheme.primary,
      ),
    ],
  ),
],
),
),
);
},
childCount: expenses.length,
),
),
),
],
),
);

```

```

}
}

```

signup_screen.dart

```

import 'package:flutter/material.dart';
import

'package:firebase_auth/firebase_auth.dart';
import
'package:firebase_database/firebase_database.
dart';
import 'home_screen.dart';

class SignupScreen extends StatefulWidget {
  @override
  _SignupScreenState createState() =>
  _SignupScreenState();
}

class _SignupScreenState extends
State<SignupScreen> with
SingleTickerProviderStateMixin {
  final _formKey = GlobalKey<FormState>();
  final _emailController =
TextEditingController();
  final _passwordController =
TextEditingController();
  final _nameController =
TextEditingController();
  final _budgetController =
TextEditingController();
  late AnimationController
_animationController;
  late Animation<double> _fadeAnimation;
  bool _isLoading = false;
  bool _obscurePassword = true;

  @override
  void initState() {
    super.initState();
    _animationController =
AnimationController(
      vsync: this,

```

```

      duration: const Duration(milliseconds: 800),
    );
    _fadeAnimation = Tween<double>(begin:
0.0, end: 1.0).animate(
      CurvedAnimation(parent:
_animationController, curve:
Curves.easeOut),
    );
    _animationController.forward();
  }

  @override
  void dispose() {
    _animationController.dispose();
    super.dispose();
  }

  Future<void> signUp() async {
    if (!_formKey.currentState!.validate())
      return;

    setState(() => _isLoading = true);
    try {
      UserCredential userCredential = await
FirebaseAuth.instance
        .createUserWithEmailAndPassword(
          email: _emailController.text.trim(),
          password: _passwordController.text);

      await FirebaseDatabase.instance

.ref('user_profiles/${userCredential.user!.uid}
')
        .set({
          'name': _nameController.text,
          'budgetGoal': _budgetController.text,
          'createdAt':
DateTime.now().toIso8601String(),
        });

      if (mounted) {
        Navigator.of(context).pushReplacement(
          PageRouteBuilder(
            pageBuilder: (context, animation,
secondaryAnimation) => HomeScreen(),
            transitionsBuilder: (context,
animation, secondaryAnimation, child) {

```

```
return FadeTransition(opacity:
```

```
animation, child: child);
```

```
    },
```

```
  ),
```

```
);
```

```
}
```

```
} on FirebaseAuthException catch (e) {
```

```
ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
```

```
    content: Text(e.message ?? 'Sign up
failed'),
```

```
    behavior: SnackBarBehavior.floating,
```

```
    backgroundColor:
```

```
Theme.of(context).colorScheme.error,
```

```
    shape:
```

```
RoundedRectangleBorder(borderRadius:
BorderRadius.circular(10)),
```

```
  ));
```

```
  } finally {
```

```
    if (mounted) setState(() => _isLoading =
false);
```

```
  }
```

```
}
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Scaffold(
```

```
    body: Container(
```

```
      decoration: BoxDecoration(
```

```
        gradient: LinearGradient(
```

```
          begin: Alignment.topLeft,
```

```
          end: Alignment.bottomRight,
```

```
          colors: [
```

```
Theme.of(context).colorScheme.primaryCont
ainer.withOpacity(0.3),
```

```
Theme.of(context).colorScheme.surface,
```

```
    ],
```

```
  ),
```

```
),
```

```
child: SafeArea(
```

```
child: Stack(
```

```
  children: [
```

```
SingleChildScrollView(
```

```
  padding: const EdgeInsets.all(24),
```

```
  child: FadeTransition(
```

```
    opacity: _fadeAnimation,
```

```
    child: Column(
```

```
      crossAxisAlignment:
```

```
CrossAxisAlignment.stretch,
```

```
      children: [
```

```
        const SizedBox(height: 40),
```

```
        Icon(
```

```
Icons.account_balance_wallet_outlined,
```

```
        size: 64,
```

```
        color:
```

```
Theme.of(context).colorScheme.primary,
```

```
      ),
```

```
      const SizedBox(height: 24),
```

```
      Text(
```

```
        'Join ExpenseTracker',
```

```
        style:
```

```
Theme.of(context).textTheme.headlineMediu
m?.copyWith(
```

```
          fontWeight:
```

```
FontWeight.bold,
```

```
        ),
```

```
        textAlign: TextAlign.center,
```

```
      ),
```

```
      const SizedBox(height: 8),
```

```
      Text(
```

```
        'Take control of your finances',
```

```
        style:
```

```
Theme.of(context).textTheme.bodyLarge?.co
pyWith(
```

```
          color:
```

```
Theme.of(context).colorScheme.onSurfaceVa
riant,
```

```
      ),
```

```
      textAlign: TextAlign.center,
```

```
    ),
```

```
    const SizedBox(height: 40),
```

```
    Card(
```

```
      elevation: 0,
```

```
      color:
```

```

Theme.of(context).colorScheme.surface.with
Opacity(0.8),
      child: Padding(
        padding: const
EdgeInsets.all(20),
        child: Form(

key: _formKey,
      child: Column(
        children: [
          _buildTextField(
            controller:
_nameController,
            label: 'Full Name',
            icon:
Icons.person_outline,
            validator: (v) =>
v?.isEmpty ?? true ? 'Required' : null,
          ),
          const SizedBox(height:
16),
          _buildTextField(
            controller:
_emailController,
            label: 'Email',
            icon:
Icons.email_outlined,
            keyboardType:
TextInputType.emailAddress,
            validator: (v) {
              if (v?.isEmpty ?? true)
return 'Required';
              if (!v!.contains('@'))
return 'Invalid email';
              return null;
            },
          ),
          const SizedBox(height:
16),

          _buildTextField(
            controller:
_passwordController,
            label: 'Password',
            icon: Icons.lock_outline,
            obscureText:
_obscurePassword,

```

```

validator: (v) {
  if (v?.isEmpty ?? true)
return 'Required';
  if (v!.length < 6) return
'Min 6 characters';
  return null;
},
suffixIcon: IconButton(
  icon:
Icon(_obscurePassword
?
Icons.visibility_outlined
:
Icons.visibility_off_outlined),
  onPressed: () =>
setState(() =>
_obscurePassword = !_obscurePassword),
),
),
const SizedBox(height:
16),
          _buildTextField(
            controller:
_budgetController,
            label: 'Monthly Budget
Goal',
            icon:
Icons.savings_outlined,
            keyboardType:
TextInputType.number,
            prefixText: '₹',
            validator: (v) =>
v?.isEmpty ?? true ? 'Required' : null,
          ),
        ],
      ),
    ),
  ),
const SizedBox(height: 24),

  FilledButton(
    onPressed: _isLoading ? null :
signUp,
    style: FilledButton.styleFrom(
      padding: const
EdgeInsets.all(16),

```

```

        shape:
RoundedRectangleBorder(
    borderRadius:
BorderRadius.circular(12),
    ),
    ),
    child: _isLoading
    ? const SizedBox(
    height: 20,
    width: 20,
    child:
CircularProgressIndicator(
    strokeWidth: 2,
    color: Colors.white,
    ),
    )
    : const Text('Create
Account'),
    ),
    ],
    ),
    ),
    ),
    SafeArea(
    child: IconButton(
    icon: const
Icon(Icons.arrow_back),
    onPressed: () =>
Navigator.pop(context),
    ),
    ),
    ],
    ),
    ),
    ),
    );

```

```

    }

Widget _buildTextField({
    required TextEditingController controller,
    required String label,
    required IconData icon,
    String? Function(String?)? validator,
    TextInputType? keyboardType,
    bool obscureText = false,
    Widget? suffixIcon,
    String? prefixText,
  }) {
    return TextFormField(
    controller: controller,
    obscureText: obscureText,
    keyboardType: keyboardType,
    style:
Theme.of(context).textTheme.bodyLarge,
    decoration: InputDecoration(
    labelText: label,
    prefixIcon: Icon(icon),
    prefixText: prefixText,
    suffixIcon: suffixIcon,
    floatingLabelBehavior:
FloatingLabelBehavior.auto,
    border: OutlineInputBorder(
    borderRadius:
BorderRadius.circular(12),
    ),
    ),
    validator: validator,
  );
}

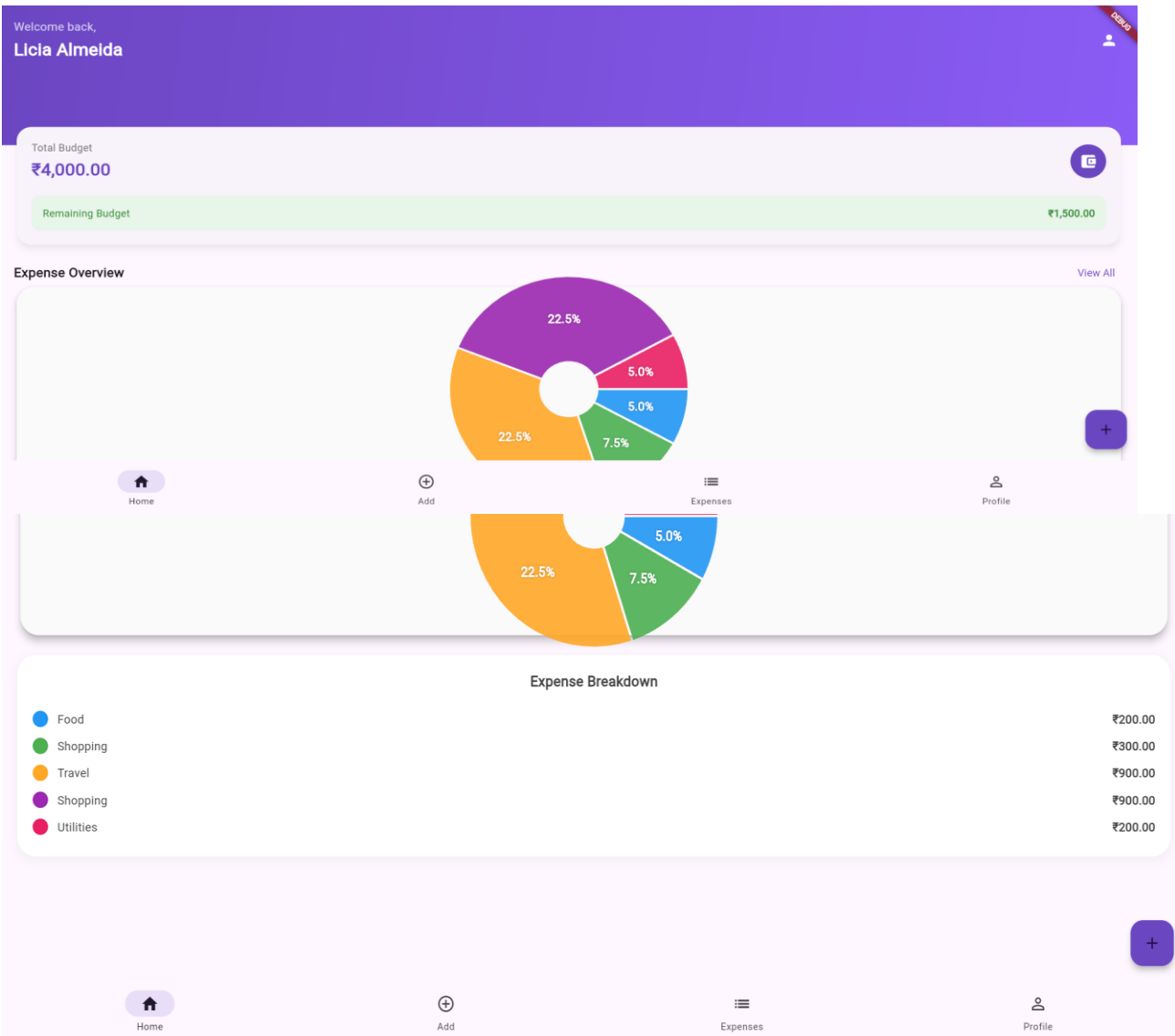
```

3.5 Final Prototype

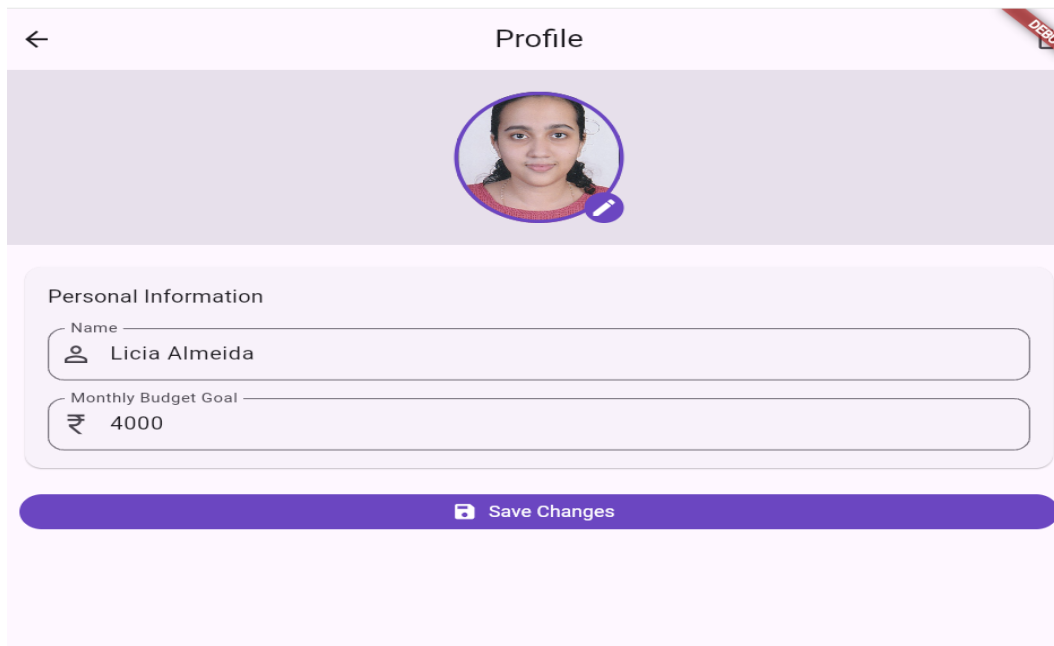
- Expense List (Fig 3)



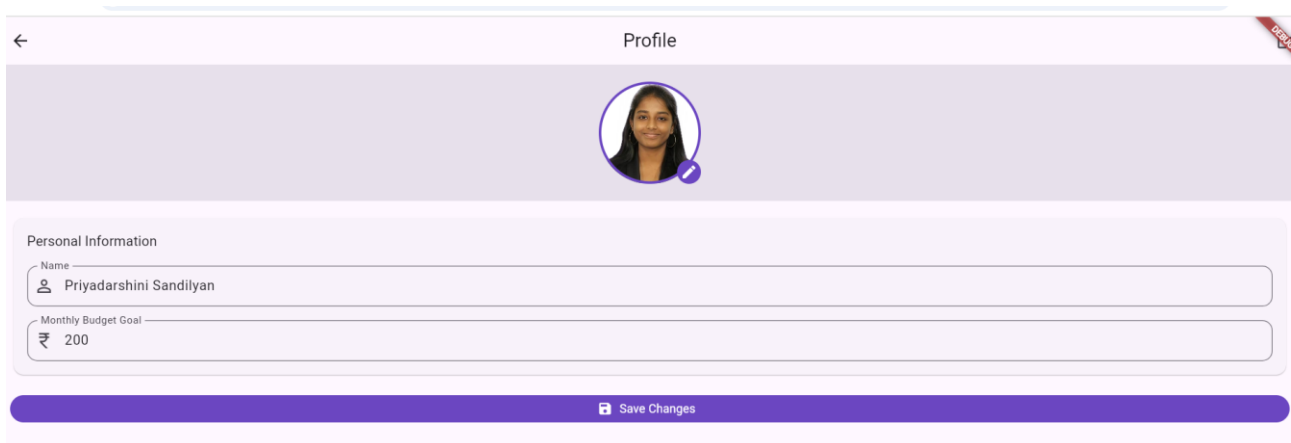
- Home_Screen (Fig 4)



- Profile Edit Page (Fig 5)

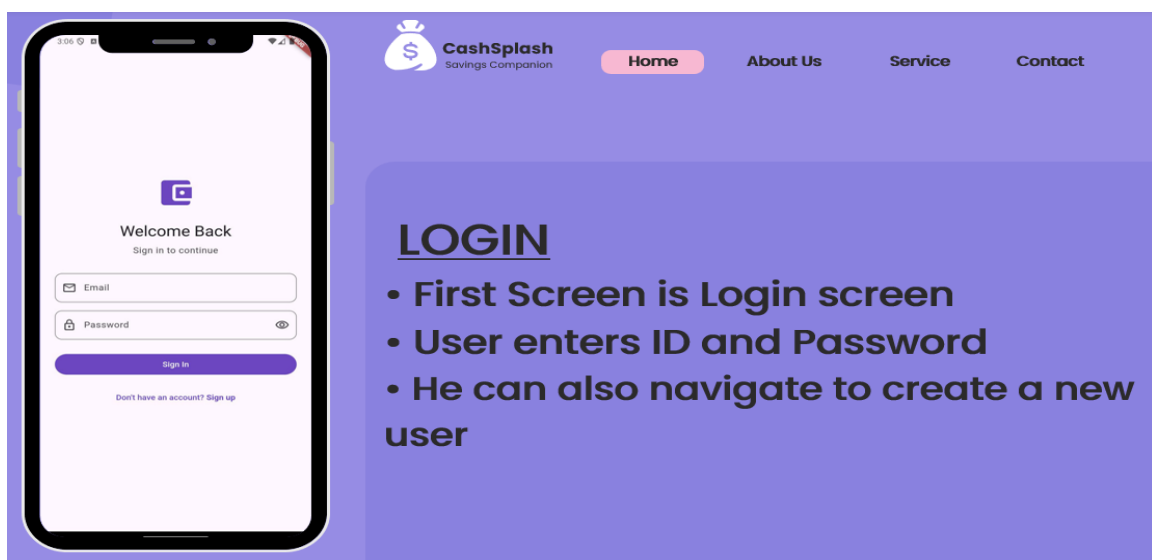


The screenshot shows a mobile app interface for editing a profile. At the top, there is a back arrow and the title "Profile". Below the title is a circular profile picture of a woman with a pencil icon indicating it can be edited. Underneath is a section titled "Personal Information" containing two input fields: "Name" with the value "Licia Almeida" and "Monthly Budget Goal" with the value "₹ 4000". At the bottom of this section is a purple button labeled "Save Changes".

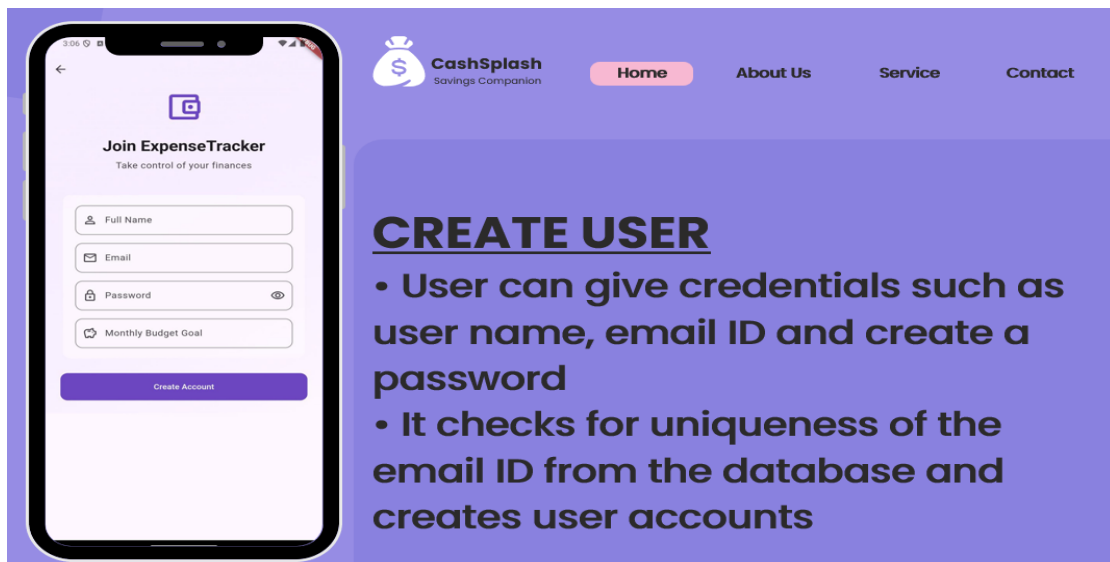


This screenshot shows the same profile edit interface as the previous one, but for a different user. The profile picture is of a woman with long dark hair. The "Personal Information" section shows the "Name" as "Priyadarshini Sandilyan" and the "Monthly Budget Goal" as "₹ 200". The "Save Changes" button is still present at the bottom.

- Login Page (Fig 6)



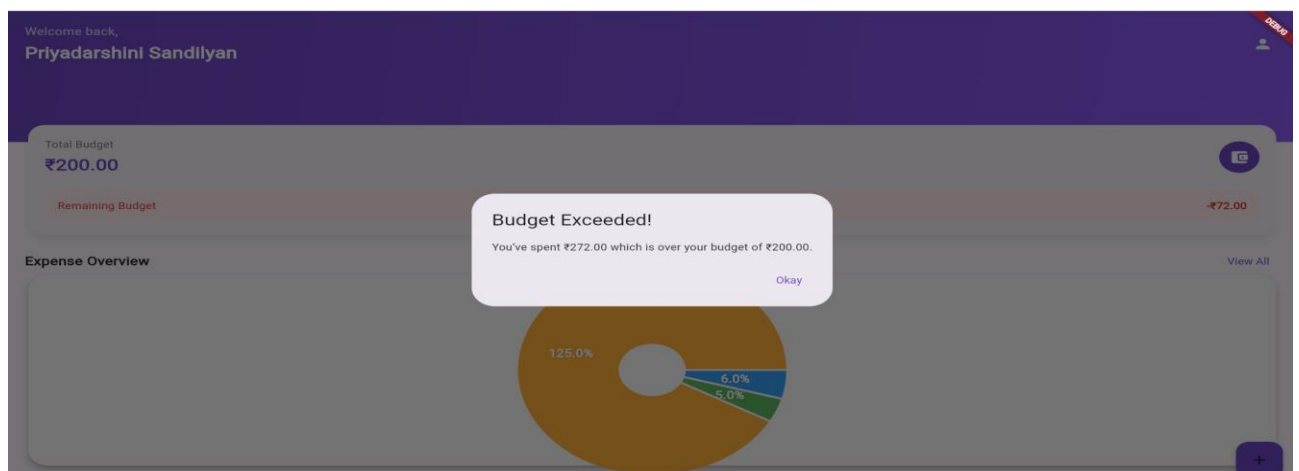
The design for the login page is presented on a purple background. On the left, a smartphone displays the mobile app's login screen. The app screen shows a "Welcome Back" message, a "Sign in to continue" prompt, and input fields for "Email" and "Password". A "Sign in" button is at the bottom, along with a link for "Don't have an account? Sign up". On the right, the website header includes the "CashSplash Savings Companion" logo and navigation links: "Home", "About Us", "Service", and "Contact". Below the header, the word "LOGIN" is written in large, bold, underlined letters. To the right of "LOGIN", three bullet points describe the login process: "• First Screen is Login screen", "• User enters ID and Password", and "• He can also navigate to create a new user".



- Expense Entry page (Fig 7)

The 'Add Expense' page features a purple header with a back arrow and the title 'Add Expense'. Below the header, the 'New Expense Details' section includes three input fields: 'Amount' (with a rupee symbol), 'Category' (with a dropdown arrow), and 'Receipt Image' (with an 'Add Receipt' button). At the bottom, there is a large purple button labeled 'Save Expense'.

- Notification For Exceeding Budget: (Fig 8)



3.6 Conclusion and Future Scope of the Project

Conclusion:

The **CashSplash** app effectively fulfills the need for a modern, personalized, and user-friendly financial tracking tool for mobile platforms. It allows users to securely register, set monthly budget goals, log daily expenses, and track their remaining budget. With a visual expense breakdown, budget alerts, and receipt uploads, the app encourages better financial awareness and planning. Built using Flutter and Firebase, the app is scalable, responsive, and easy to use even for non-technical users.

Future Scope:

- AI-based financial insights and budgeting suggestions
- Integration with UPI or bank accounts for automated transaction logging
- Multi-user budgeting and collaborative financial planning (for families or roommates)
- Smart notifications and reminders for recurring expenses and bill payments
- Export reports or download monthly spending summaries as PDFs

3.7 Constraints for Real-Time Deployment

- Dependence on internet connectivity for Firebase real-time data sync
- Security and privacy risks in handling sensitive financial data
- Firebase's free-tier limitations on authentication, storage, and database usage
- Limited compatibility with older Android devices or absence of iOS support
- Currently lacks integration with official banking APIs or payment gateways
- No biometric authentication yet (face/fingerprint login)

4. References

1. Firestore Documentation – Authentication and Realtime Database
2. Flutter Documentation – Widgets, Navigation, and State Management
3. Android Developers Guide – <https://developer.android.com>
4. YouTube Channels :
 - 1) <https://youtu.be/uWRtXmll8yo?si=z4wY2VVRDrQ2qCJF>
 - 2) <https://www.youtube.com/watch?v=ncvni8ZiQ3M&list=PLFMcmbNqJmhoLgUwcCvoGbX0TIqgKl8bY&index=2>